

ADMINISTRATION ET MAINTENANCE LINUX 2

CHAPITRE 1 : LES SYSTEME D'EXPLOITATION LINUX

- 1.1 Introduction
- 1.2 Choisir une distribution
 - 1.2.1 Debian
 - 1.2.2 Ubuntu
 - 1.2.3 Red Hat et Fedora
 - 1.2.4 Mandriva (exMandrake)
 - 1.2.5 openSUSE

CHAPITRE 2 LE SHELL PAR DEFAULT : BASH

- 2.1. L'invite de commande
- 2.2 Utiliser le shell
- 2.3 La gestion des fichiers
- 2.4 Les chemins
- 2.5 Lister les fichiers et les répertoires
- 2.6 Les liens symboliques
- 2.7. Wildcards : caractères de substitution

CHAPITRE 3 : RECHERCHER DES FICHIERS

- 3.1. Considérations générales
- 3.2. Critères de recherche
- 3.3 Afficher le contenu d'un fichier : cat & less
 - 3.3.1 La commande cat
 - 3.3.2 La commande less
 - 3.3.3 Les raccourcis basiques indispensables
 - 3.3.4 afficher le début et la fin d'un fichier : head & tail

CHAPITRE 4 : ÉDITEURS DE TEXTE

- 4.1. Nano, l'éditeur de texte du débutant
 - 4.1.1 Nano est un éditeur de texte, pas un traitement de texte !
 - 4.1.2 Découverte de Nano
 - 4.1.3 Les paramètres de la commande Nano

CHAPITRE 5 : LES UTILISATEURS ET LES DROITS

- 5.1. sudo : exécuter une commande en root
- 5.2. Gestion des utilisateurs : adduser
 - 5.2.1 Ajouter un utilisateur : adduser
 - 5.2.2 changer le mot de passe : passwd
 - 5.2.3 supprimer un compte : deluser
- 5.3. Modifier les droits d'accès : chmod
 - 5.3.1 Le fonctionnement des droits
 - 5.3.2 Modifier les droits d'accès : chmod

CHAPITRE 6 : LES FLUX DE REDIRECTION

- 6.1 Rediriger le résultat dans un fichier : > et >>
 - 6.1.1. rediriger dans un nouveau fichier : >
 - 6.1.2 rediriger à la fin d'un fichier : >>

6.1.3 rediriger les erreurs : 2>, 2>> et 2>&1

6.2.1 le pipe |

CHAPITRE 7 : AUTOMATISATION DES TACHES

7.1 Exécuter un programme a une heure différée

7.2 Exécuter une commande après un certain délai

7.3 lister et supprimer les *jobs* en attente : atq et atrm

7.4 sleep : faire une pause

7.5 Exécuter une commande régulièrement : crontab

7.6 Automatisez vos tâches avec des scripts Bash

CHAPITRE 8 : LA CONNEXION SECURISEE A DISTANCE AVEC SSH

8.1. Introduction

8.2 De Telnet à SSH

8.3 Se connecter avec SSH et PuTTY

8.3.1 Transformer sa machine en serveur

8.3.1 Se connecter via SSH à partir d'une machine Linux

8.3.2 Se connecter via SSH à partir d'une machine Windows

CHAPITRE 9 : TRANSFERER DES FICHIERS

9.1 Téléchargement de fichiers : wget

9.2 scp : copier des fichiers sur le réseau

9.3 Copier un fichier de votre ordinateur vers un autre

9.4 Copier un fichier d'un autre ordinateur vers le vôtre

9.5 ftp & sftp : transférer des fichiers

9.6 Le transfert de fichiers

9.7. un FTP sécurisé : sftp

9.8 Synchroniser des fichiers pour une sauvegarde : rsync

9.9 Sauvegarder dans un autre dossier du même ordinateur

9.10 Sauvegarder sur un autre ordinateur

CHAPITRE 10 : ANALYSER LE RESEAU ET FILTRER LE TRAFIC AVEC UN PARE-FEU

10. 1 iptables : le pare-feu de référence

10.2. Afficher les règles : *iptables -L*

10.3 Ajouter et supprimer des règles

10.4 Autoriser les pings

10.5 Autoriser les connexions locales et déjà ouvertes

10.6 Refuser toutes les autres connexions par défaut

CHAPITRE 11 : SERVICES RESEAUX ET DEPANNAGE

11.1 Configuration

11.2. Les services

11.3 Les protocols: /etc/protocols

11.4 Services réseaux xinetd

11.5 Monter un serveur DHCP

11.6 Serveur DNS

11.7 Service HTTP Apache

11.8 Problèmes de maintenance

CHAPITRE 1

LES SYSTEME D'EXPLOITATION LINUX

1.1 Introduction

Un système d'exploitation est un programme ou un ensemble de programmes assurant la gestion de l'ordinateur et des périphériques. Il sert d'interface entre le matériel (hardware) et le logiciel (software). C'est un ensemble de programmes très complexes dont le but est de rendre plus simples les programmes et l'utilisation de l'ordinateur. Le système d'exploitation propose aux programmeurs une interface de programmation d'applications appelée **API**, *Application Programming Interface*. Tous les programmeurs utilisent les mêmes fonctions dans leurs programmes ce qui simplifie fortement le travail. Ils peuvent se concentrer sur le but de leur programme (créer un traitement de texte par exemple) sans avoir sans arrêt à écrire des morceaux de programmes pour gérer le disque dur, l'imprimante ou comment accéder au clavier. C'est le rôle du système d'exploitation de gérer : la mémoire, les accès aux périphériques, les données sur les disques, les programmes, la sécurité, la collecte des informations.

Il manque l'interface graphique. Dans un produit comme Microsoft Windows l'interface graphique est incluse au sein même du système d'exploitation. Il est d'ailleurs impossible de travailler sans, le moindre réglage se fait depuis une boîte de dialogue. Les utilisateurs ont de ce fait tendance à intégrer l'interface graphique comme composant de tout système d'exploitation. Historiquement l'interface graphique ne fait pas partie du système d'exploitation. Elle vient en complément. Vous ne trouverez aucun livre sur la théorie des systèmes d'exploitation traitant des interfaces graphiques. L'interface graphique n'est pas un composant du système d'exploitation Linux qui n'en a pas besoin pour fonctionner correctement. C'est un ensemble de plusieurs programmes classiques exécutés au-dessus du système d'exploitation, qu'il utilise.

Linux est un système d'exploitation de type Unix. Il existe des dizaines de systèmes d'exploitation dans cette famille. Unix est un système d'exploitation de la famille des systèmes **multitâches** et **multiutilisateurs** :

- **Multitâche** : le système gère l'exécution simultanée de plusieurs programmes appelés des processus
- **Multiutilisateurs** : le système permet l'existence de plusieurs utilisateurs différents sur une même machine, connectés ou non (un utilisateur peut faire tourner un programme sans être connecté, comme par exemple un serveur Internet).
- Les **appels systèmes** sont utilisés par les programmes pour communiquer avec le système d'exploitation Unix.

- La **gestion des processus** s'occupe de la commutation des tâches et de leur priorité. Ce composant s'occupe donc du multitâche.
- La **gestion des entrées et des sorties fichiers** s'occupe aussi bien de la lecture et de l'écriture des données sur vos disques durs mais aussi sur vos périphériques (carte son, imprimante, etc.)
- Certaines informations peuvent être placées dans une zone mémoire tampon appelée **cache**. Plutôt que d'écrire des données directement sur le disque dur (ce qui est lent), Unix va les écrire dans une zone mémoire puis ensuite les écrire sur le disque après quelques secondes. Ainsi, la relecture de ces données est plus rapide car elles sont déjà en mémoire et le logiciel ne perd pas de temps à attendre la fin de l'écriture des données.
- Les **pilotes** ont pour rôle de gérer au plus bas niveau le matériel ou les structures logiques du matériel (par exemple les données d'une partition).

1.2 Choisir une distribution

Les années 1994-1997 voient l'apparition des grandes distributions Linux que vous connaissez encore aujourd'hui : Red Hat, Debian, Suse, Slackware. Mandrake est arrivé un peu plus tard. C'est surtout durant ces années que Linux sort du petit monde des hackers et se fait connaître en entreprise.

1.2.1 Debian

Debian a longtemps été la seule distribution entièrement et uniquement composée de logiciels libres et Open Source.

Les avantages de Debian sont nombreux :

- un nombre gigantesque de packages qui se chiffre en milliers,
- un logiciel d'installation appelé APT très pratique et performant,
- une distribution 100% open source,
- une stabilité à toute épreuve pour un environnement de production.

Ces avantages entraînent aussi quelques inconvénients :

- des packages souvent anciens,
- des mises à jour de la distribution irrégulières et trop espacées,
- des risques liés à la multiplication des paquets et des dépendances,
- une installation et une configuration compliquées.



Tous ces inconvénients ne sont pas forcément des défauts. Faut-il préférer une version ancienne mais totalement exempte de bugs ou la toute dernière version d'un produit dont la fiabilité n'a pas été pleinement éprouvée ?

1.2.2 Ubuntu

Devant les quelques inconvénients de la distribution le milliardaire sudafricain Mark Shuttleworth crée la distribution Ubuntu Linux en 2005.

- une distribution issue de Debian,
- une compatibilité avec les packages de Debian,
- un système d'installation très simple,
- une sortie tous les 6 à 8 mois,
- un environnement graphique agréable.

Cette distribution est idéale pour les étudiants, cependant la tentation est très forte de revenir au fonctionnement d'une distribution Debian, les deux étant compatibles.



1.2.3 Red Hat et Fedora

S'il y a bien une société commerciale dans le monde Linux qui a marqué et qui continue à marquer son époque, c'est bien la société Red Hat. Les distributions Red Hat ont très fortement marqué les esprits car elles sont restées la référence pendant presque dix ans. Red Hat se concentre maintenant sur le monde de l'entreprise avec des distributions commerciales appelées **RHEL** (*Red Hat Enterprise Linux*) :

- des versions professionnelles destinées aux entreprises,
- des solutions du poste de travail au plus gros serveur,
- des architectures matérielles nombreuses,
- un support commercial,
- des mises à jour assurées pendant sept ans,
- 100% libre.



1.2.4 Mandriva (exMandrake)

Mandriva Linux (exMandrake) est une distribution dérivée et longtemps entièrement compatible avec la distribution Red Hat. Mandriva continue d'innover fortement, notamment dans le poste de travail nomade avec des distributions clé en main bootables depuis des clés USB.



1.2.5 openSUSE

Se prononçant *sousse*, **openSUSE** est une distribution d'origine allemande datant de 1992. Le nom de l'entreprise lui-même était un hommage au célèbre **Konrad Zuse** l'inventeur des ordinateurs modernes.



CHAPITRE 2

LE SHELL PAR DEFAULT : BASH

2.1. L'invite de commande

Le shell attend des entrées au clavier sur une ligne appelée l'invite de commande ou prompt. Un curseur, qui peut être représenté par un rectangle fixe, clignotant ou un caractère souligné, indique la position actuelle de votre saisie. L'invite (prompt) fournit des informations sur le terminal et votre position dans le système de fichiers.

```
seb@slyserver:/home/public>
```

Dans cette invite tout à fait classique, vous trouvez quatre informations :

- **seb** : c'est le nom de connexion, ou login de l'utilisateur, actuellement connecté au terminal ;
- **slyserver** : c'est le nom d'hôte (hostname), le nom logique de la machine raccordée au terminal ;
- **/home/public** : c'est la position actuelle du shell dans le système de fichiers ;
- **>** : c'est la terminaison standard du bash pour un utilisateur sans pouvoirs.

Cette invite vous informe que c'est l'utilisateur sans pouvoirs d'administration seb qui utilise le terminal (est connecté) sur la machine slyserver et qu'il est actuellement positionné dans /home/public. Le caractère de terminaison peut avoir d'autres significations :

- un **\$** indique que l'utilisateur n'a pas de pouvoirs particuliers, comme le **>**.
- un **#** indique que l'utilisateur est l'administrateur root qui a tous les pouvoirs.

Le chemin peut varier :

- **seb@slyserver:~>** : le caractère tilde ~ indique que vous êtes dans votre répertoire personnel.

- **seb@slyserver:~/test>** : le ~ se rapportant à votre répertoire personnel, vous êtes dans le répertoire test au sein de celui-ci.

Dans la suite, l'invite de commande sera généralement remplacé par un simple dollar, \$, ceci afin de gagner de la place sur la ligne de commande.

2.2 Utiliser le shell

- La saisie

Dans le terminal, le clavier s'utilise comme d'habitude. Vous pouvez vous déplacer sur la ligne avec les flèches de droite et de gauche du clavier et effacer des caractères avec les touches [Retour arrière] et [Suppr]. Vous lancez l'exécution de la commande que

vous avez saisi en appuyant sur la touche [Entrée]. Il est temps de tester quelques commandes. La commande **date** indique la date et l'heure actuelles.

```
$ date
```

```
Tue Oct 29 06:18:43 AM EDT 2024
```

Une commande pratique, **pwd**, permet de savoir à quel endroit vous vous situez dans les répertoires.

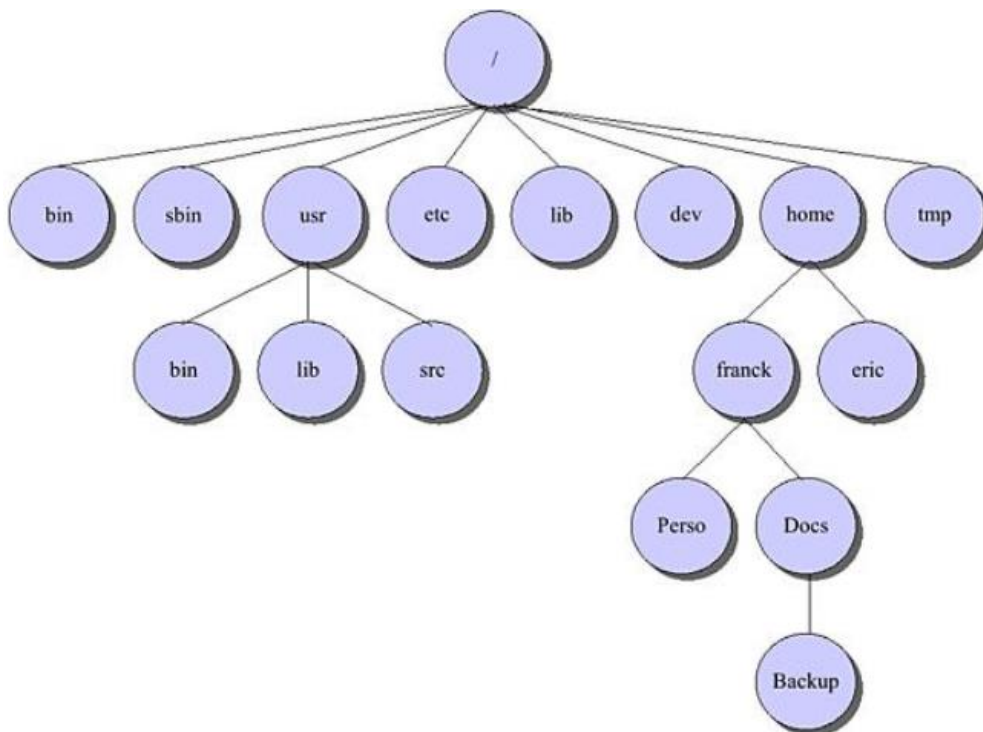
```
$ pwd
```

```
/home/seb
```

2.3 La gestion des fichiers

2.3.1. Le système de fichiers

Un système de fichiers, appelé communément File System ou FS, définit l'organisation des données sur un support de stockage, donc comment sont gérés et organisés les fichiers par le système d'exploitation. Linux est un système d'exploitation entièrement orienté fichier. Tout (ou presque) est représenté par un fichier, tant les données (fichiers de données de tout type comme une image ou un programme), que les périphériques (terminaux, souris, clavier, carte son, etc.) ou encore les moyens de communication (sockets, tubes nommés, etc.). On peut dire que le système de fichier est le coeur de tout système Unix.



Le système de fichiers de Linux est hiérarchique. Il décrit une arborescence de répertoires et de sous-répertoires, en partant d'un élément de base appelé la racine ou root directory.

2.3.2 Les divers types de fichiers

On distingue trois types de fichiers : ordinaires, catalogue, spéciaux.

a. Les fichiers ordinaires ou réguliers

Les fichiers ordinaires sont aussi appelés fichiers réguliers, ordinary files ou regular files. Ce sont des fichiers tout à fait classiques qui contiennent des données. Par données, comprenez n'importe quel contenu :

- texte ; image ; audio ; programme binaire compilé ; script ; base de données ; bibliothèque de programmation, etc.

b. Les catalogues

Les fichiers catalogues sont les répertoires, dossiers ou directory. Les répertoires permettent d'organiser le disque dur en créant une hiérarchie. Un répertoire peut contenir des fichiers normaux, des fichiers spéciaux et d'autres répertoires, de manière récursive. Un répertoire n'est rien d'autre qu'un fichier particulier contenant la liste des fichiers eux-mêmes présents dans ce répertoire. Cette notion se révélera très utile lorsque la question des droits sera abordée.

c. Les fichiers spéciaux

Le troisième type de fichier est le fichier spécial. Il existe plusieurs genres de fichiers spéciaux. Ils se trouvent principalement dans le répertoire /dev s'ils représentent des périphériques. Ce sont principalement des fichiers servant d'interface pour les divers périphériques. Ils peuvent s'utiliser, suivant le cas, comme des fichiers normaux.

2.3.3 Nomenclature des fichiers

On ne peut pas donner n'importe quel nom à un fichier, il faut pour cela suivre quelques règles simples. Ces règles sont valables pour tous les types de fichiers. Sur les anciens systèmes Unix un nom de fichier ne pouvait pas dépasser 14 caractères. Sur les systèmes actuels, dont Linux, on peut aller jusqu'à 255 caractères. L'éventuelle extension est comprise dans la longueur du nom du fichier. Un point extrêmement important : Linux fait la distinction entre les noms de fichiers en minuscules et en majuscules. Toto, TOTO, ToTo et toto sont des noms de fichiers différents, avec un contenu différent. La plupart des caractères (les chiffres, les lettres, les majuscules, les minuscules, certains signes, les caractères accentués) sont acceptés, y compris l'espace. Cependant quelques caractères sont à éviter car ils ont une signification particulière au sein du shell : & ; () ~ <espace> \ / | ` ? (en début de nom).

Les noms suivants sont valides :

- Fichier1, Paie.txt, 123traitement.sh, Paie_juin_2002.xls, 8

Ces noms peuvent poser des problèmes :

- Fichier*, Paie(decembre), Ben&Nuts, Paie juin 2002.xls

2.4 Les chemins

2.4.1 Structure et nom de chemin

Les chemins permettent de définir un emplacement au sein du système de fichiers. C'est la liste des répertoires et sous-répertoires empruntés pour accéder à un endroit donné de l'arborescence jusqu'à la position souhaitée (répertoire, fichier). Un nom de fichier est ainsi généralement complété par son chemin d'accès. C'est ce qui fait que le fichier toto du répertoire rep1 est différent du fichier toto du répertoire rep2.

Le schéma présenté dans la section La gestion des fichiers, système de fichiers de ce chapitre représente une arborescence d'un système de fichier Linux. Le / situé tout en haut s'appelle la racine ou root directory (à ne pas confondre avec le répertoire de l'administrateur root). Le nom de chemin ou path name d'un fichier est la concaténation, depuis la racine, de tous les répertoires qu'il est nécessaire de traverser pour y accéder, chacun étant séparé par le caractère /. C'est un chemin absolu comme celui-ci.

/home/toto/Docs/Backup/fic.bak

2.4.2 Un chemin absolu ou complet

Un chemin absolu :

- démarre de la racine, donc commence par un /
- décrit tous les répertoires à traverser pour accéder à l'endroit voulu,
- ne contient pas de . ni de ..

2.4.2 Chemin relatif

Un nom de chemin peut aussi être relatif à sa position courante dans le répertoire. Le système (ou le shell) mémorise la position actuelle d'un utilisateur dans le système de fichier, le répertoire actif. Vous pouvez accéder à un autre répertoire de l'arborescence depuis l'emplacement actuel sans taper le chemin complet uniquement en précisant le chemin le plus court relativement à votre position actuelle au sein de l'arborescence.

Il faut pour cela souvent utiliser deux entrées particulières de répertoires :

- Le point . représente le répertoire courant, actif. Il est généralement implicite.
- Les doubles points .. représentent le répertoire de niveau inférieur.

Un chemin relatif :

- décrit un chemin relatif à une position donnée dans l'arborescence, généralement (mais pas toujours) depuis la position courante ;
- décrit en principe le plus court chemin pour aller d'un point à un autre ;
- peut contenir des points ou des doubles points ;

Ces trois affirmations ne sont pas des obligations :

- `/usr/local/bin` est un chemin complet ou absolu ;
- `Documents/Photos` est un chemin relatif : le répertoire `Documents` est considéré comme existant dans le répertoire courant ;
- `./Documents/Photos` est un chemin relatif parfaitement identique au précédent, sauf que le répertoire actif (courant) est explicitement indiqué par le point. « `./Documents` » indique explicitement le répertoire `Documents` dans le répertoire actif ;
- `/usr/local/../bin` est un chemin relatif : les `..` sont relatifs à `/usr/local` et descendent d'un niveau vers `/usr`. Le chemin final est donc `/usr/bin`.

2.4.3 Répertoire personnel

Lors de la création d'un utilisateur, l'administrateur lui alloue un répertoire personnel appelé `home directory`. Lorsqu'il se connecte, l'utilisateur arrive directement dans ce répertoire, qui est son répertoire personnel. C'est dans ce répertoire que l'utilisateur pourra créer ses propres fichiers et répertoires.

`$ pwd`

`/home/seb`

La commande `cd` permet de rentrer dans le répertoire personnel quel que soit l'endroit où on se trouve dans le système de fichier.

2.5 Lister les fichiers et les répertoires

La commande `ls` permet de lister le contenu d'un répertoire (catalogue) en lignes ou colonnes. Elle supporte plusieurs paramètres dont voici les plus pertinents.

Paramètre	Signification
<code>-l</code>	Pour chaque fichier ou dossier, fournit des informations détaillées.
<code>-a</code>	Les fichiers cachés sont affichés (ils commencent par un point).
<code>-d</code>	Sur un répertoire, précise le répertoire lui-même et non son contenu.
<code>-F</code>	Rajoute un caractère à la fin du nom pour spécifier le type : <code>/</code> pour un répertoire, <code>*</code> pour un exécutable, <code>@</code> pour un lien symbolique, etc.

-R	Si la commande rencontre des répertoires, elle rentre dans les sous-répertoires, soussous-répertoires, etc., de manière récursive.
----	--

2.5.1 Gérer les fichiers et les répertoires

a. Créer des fichiers

Pour vos tests ou durant vos actions vous pouvez avoir besoin de créer des fichiers vides. Une commande pratique pour cela est **touch**. Utilisée avec uniquement le nom d'un fichier en argument, elle crée un fichier avec une taille nulle.

\$ touch fichier

Créer plusieurs fichiers à la fois

\$ touch fichier1 fichier2 fichier3

b. Créer des répertoires

La commande **mkdir** (*make directory*) permet de créer un ou plusieurs répertoires, ou une arborescence complète. Par défaut la commande ne crée pas d'arborescence. Si vous passez comme arguments rep1/rep2 et que rep1 n'existe pas, la commande retourne une erreur. Dans ce cas, utilisez le paramètre -p.

\$ mkdir Documents

\$ mkdir Documents/Photos

\$ mkdir -p Archives/vieilleries

c. Supprimer des répertoires

La commande **rmdir** (*remove directory*) supprime un ou plusieurs répertoires. Elle ne peut pas supprimer une arborescence. Si des fichiers sont encore présents dans le répertoire, la commande retourne une erreur. Le répertoire ne doit donc contenir ni fichiers ni répertoires et ceci même si les sous-répertoires sont eux-mêmes vides.

Pour supprimer une arborescence, utiliser la commande **rm** avec l'option **-r**

\$ rm -r dossier

d. Copier des fichiers

La commande **cp** (*copy*) copie un ou plusieurs fichiers vers un autre fichier ou vers un répertoire. *\$ cp fichier fichiercopie*

Le premier paramètre est le nom du fichier à copier, le second le nom de la copie du fichier à créer. En faisant cela, on aura donc deux fichiers identiques dans le même répertoire : fichier et fichiercopie.

e. Copier dans un autre dossier

```
$ cp fichierbidon mondossier /
```

Le fichier fichierbidon sera copié dans mondossier sous le même nom. Si vous voulez copier fichierbidon dans mondossier sous un autre nom, faites comme ceci :

```
$ cp fichierbidon mondossier / fichiercopie
```

La commande peut prendre, entre autres, les options suivantes :

Paramètre	Signification
-r	Récuratif : copie un répertoire et tout son contenu.
-p	Les permissions et dates sont préservées.
-f	Forcer la copie.

Votre attention doit être attirée sur le fonctionnement de **cp** avec les copies de répertoires. Le fonctionnement est différent selon la présence du répertoire de destination ou non. Dans le cas où rep2 n'existe pas, le répertoire rep1 est copié en rep2. À la fin rep2 est une copie exacte de rep1.

```
$ cp -r rep1 rep2
```

```
$ ls
```

```
rep1 rep2
```

Maintenant que rep2 existe, exécutez de nouveau la commande cp. Cette fois, comme rep2 existe, il n'est pas écrasé comme vous pourriez le penser. La commande détermine que la destination étant le répertoire rep2, rep1 doit être copiée dans la destination : rep1 est copié dans rep2.

```
$ cp -r rep1 rep2
```

```
$ ls rep2
```

```
rep1
```

f. Utiliser le joker *

Le symbole * est appelé *joker*, ou encore *wildcard* en anglais sous Linux. Il vous permet de copier par exemple tous les fichiers image .jpg dans un sous-dossier :

2.5.2 Déplacer et renommer un fichier

La commande **mv** (*move*) permet de déplacer, de renommer un fichier, ou les deux en même temps. Elle fonctionne comme la commande cp. Avec les trois commandes mv successives suivantes :

```
$ touch txt1 txt2 txt3
```

```
$ mv txt1 txt1.old
```

```
$ mv txt2 rep1/txt2
```

```
$ mv txt3 rep1/txt3.old
```

- txt1 est renommé en txt1.old ;
- txt2 est déplacé dans rep1 ;
- txt3 est déplacé dans rep1 et renommé en txt3.old

2.5.3 Supprimer un fichier ou une arborescence

La commande **rm** (*remove*) supprime un ou plusieurs fichiers, et éventuellement une arborescence complète, suivant les options. **La suppression est définitive.** Les options sont classiques mais vu la particularité et la dangerosité de la commande il est bon de faire un rappel.

Paramètre	Signification
i	La commande demandera une confirmation pour chacun des fichiers à supprimer. Suivant la version d'Unix, le message change et la réponse aussi : y, Y, O, o, N, n, parfois toutes.
r	Le paramètre suivant attendu est un répertoire. Dans ce cas, la suppression est récursive : tous les niveaux inférieurs sont supprimés, les répertoires comme les fichiers.
f	Force la suppression.

L'utilisation combinée des paramètres -r et -f bien que très utile et pratique est très dangereuse, notamment en tant que root. Aucune confirmation ne vous est demandée. À moins d'utiliser des outils de récupération de données spécifiques, chers et peu performants, vos données sont irrémédiablement perdues.

2.5.4 rm et le joker de la mort (qui tue)

Je vais vous montrer quelque chose d'horrible, d'affreux, d'interdit aux moins de 18 ans, bref vous m'avez compris, le truc à ne faire sous aucun prétexte, même pas en cauchemar.

NON NON NON NE FAITES JAMAIS CA !!! => rm -rf /*

Je me suis permis de mettre du texte avant pour vous éviter la tentation de recopier bêtement la commande pour « rigoler », pour « voir ce que ça fait ». Je vais vous l'expliquer dans le détail, parce que c'est quand même **l'erreur n° 1** à ne pas faire sous Linux.

- rm : commande la suppression ;
- r : supprime de manière récursive tous les fichiers et dossiers ;
- f : force la suppression sans demander la moindre confirmation ;

– /* : supprime tous les fichiers et dossiers qui se trouvent à la racine (/) quel que soit leur nom (joker *).

En clair, cette commande supprime tout votre disque dur depuis la racine, sous-dossiers compris, et ne demande aucune confirmation. Aucune possibilité de récupération, votre PC est foutu. Vous êtes bons pour une réinstallation de Linux, **et aussi de Windows** si la partition de Windows était accessible depuis Linux

Le joker reste quand même très utile, mais lorsque vous l'utilisez avec rm, triplez d'attention. Par exemple :

```
$ rm -rf *
```

2.6 Les liens symboliques

Vous pouvez créer des liens, qui sont un peu comme des raccourcis. Un lien est un fichier spécial contenant comme information un chemin vers un autre fichier. C'est une sortie d'alias. Il existe deux types de liens : le lien dur (hard link), et le lien symbolique (soft link) qui correspond à la définition donnée.

```
$ ln -s fichier lien
```

Le cas échéant le lien se comportera à l'identique du fichier pointé avec les mêmes permissions et les mêmes propriétés :

- si le fichier pointé est un programme, lancer le lien lance le programme ;
- si le fichier pointé est un répertoire, un cd sur le lien rentre dans ce répertoire ;
- si le fichier pointé est un fichier spécial (périphérique), le lien est vu comme périphérique ; etc.

Si vous disposez d'un terminal en couleur, il est possible que le lien symbolique apparaisse, par convention sous Linux, en bleu ciel. S'il apparaît en rouge, c'est qu'il pointe dans le vide. Ce n'est pas parce qu'un lien pointe dans le vide qu'il est forcément mauvais. C'est peut-être fait exprès car il est possible de créer des liens vers des clés USB, des CDROMs, entre divers systèmes de fichiers, qui peuvent être amovibles. Dans ce cas, le lien redevient actif quand le support est inséré et/ou que la cible est de nouveau présente.

2.7. Wildcards : caractères de substitution

Lors de l'utilisation de commandes en rapport avec le système de fichier, il peut devenir intéressant de filtrer la sortie de noms de fichiers à l'aide de certains critères, par exemple avec la commande **ls**. Au lieu d'afficher toute la liste des fichiers, on peut filtrer

l'affichage à l'aide de divers critères et caractères spéciaux. Créer les fichiers suivants :

afic afic2 bfic bfic2 cfic cfic2 dfic dfic2 afic1 afic3 bfic1 bfic3 cfic1 cfic3 dfic1 dfic3

- Pour obtenir tous les fichiers commençant par a :

```
$ ls a*
```

afic1 afic2 afic3

- Tous les fichiers de quatre caractères commençant par a :

```
$ ls a???
```

afic

- Tous les fichiers d'au moins trois caractères et commençant par b :

```
$ ls b??*
```

bfic bfic1 bfic2 bfic3

- Tous les fichiers finissant par 1 ou 2 :

```
$ ls *[12]
```

afic1 afic2 bfic1 bfic2 cfic1 cfic2 dfic1 dfic2

- Tous les fichiers commençant par les lettres de a à c, possédant au moins un second caractère avant la terminaison 1 ou 2 :

```
$ ls [a-c]?*[12]
```

afic1 afic2 bfic1 bfic2 cfic1 cfic2

- Tous les fichiers ne finissant pas par 3 :

```
$ ls *[^3]
```

afic afic1 afic2 bfic bfic1 bfic2 cfic cfic1 cfic2 dfic dfic1 dfic2

CHAPITRE 3

RECHERCHER DES FICHIERS

3.1. Considérations générales

La commande **find** permet de rechercher des fichiers au sein de l'arborescence du système de fichiers à l'aide de critères et donne la possibilité d'agir sur les résultats retournés : *find chemin critères options*

La commande **find** étant réursive, il suffit d'indiquer un répertoire de base pour que toute l'arborescence depuis ce répertoire soit développée.

3.2. Critères de recherche

Les paramètres permettent de définir les critères de recherche. Ces critères, s'ils sont plusieurs, sont combinés entre eux par un ET (critère1 ET critère2).

3.2.1 name

-name permet une sélection par noms de fichiers. Il est possible d'utiliser les wildcards déjà vus. Le critère est idéalement placé entre guillemets. Ici la liste de tous les fichiers depuis l'emplacement courant et commençant par « fic » est affichée.

```
$ find . name "fic*"
```

```
./fic1
```

```
./fic2
```

```
./fic3
```

```
./fic4
```

3.2.2 type

-type permet une sélection par type de fichier. Vous savez déjà que outre les liens, les répertoires et les fichiers simples, étaient présents d'autres types de fichiers.

Code	Type de fichier
b	Fichier spécial en mode bloc
c	Fichier spécial en mode caractère
d	Répertoire (directory)
f	Fichier ordinaire
l	Lien symbolique
p	Tube nommé (pipe)
s	Socket (Connexion réseau)

Tous les répertoires dont le nom commence par « re » sont affichés.

```
$ find . -name "re*" -type d
```

```
./rep1
```

```
./rep2
```

3.2.3 size

-size permet de préciser la taille des fichiers recherchés.

- size 10k : recherche les fichiers d'une taille de 10 Ko (10*1024 octets = 10240 octets).
- size +5000k : les fichiers de plus de 5000 Ko.
- size 100k : les fichiers de moins de 100 Ko.

3.3 Afficher le contenu d'un fichier : cat & less

Nous allons voir comment afficher le contenu d'un fichier. Il y a en gros deux commandes basiques sous Linux qui permettent de faire cela : **cat** et **less**. Aucune de ces commandes ne permet d'éditer un fichier, elles permettent juste de **voir** son contenu. Pour nos exemples, nous allons travailler sur un fichier qui existe déjà : syslog. Il se trouve dans le dossier /var/log. Commencez par vous y rendre

```
$ cd /var/log
```

```
$ cat syslog
```

3.3.1 La commande cat

Permet d'afficher tout le contenu d'un fichier dans la console d'un coup. Il vous suffit d'indiquer en paramètre le nom du fichier que vous voulez afficher

```
$ cat monfichier
```

Il y a peu de paramètres vraiment intéressants à utiliser avec la commande cat, car c'est une commande somme toute très basique. On notera quand même le paramètre -n qui permet d'afficher les numéros de ligne :

```
$ cat -n monfichier
```

3.3.2 La commande less

La commande cat est rapide. Trop rapide. Tout le fichier est lu et affiché d'un coup dans la console, ce qui fait que l'on n'a pas le temps de le lire s'il est très gros. C'est là qu'une autre commande comme less devient vraiment indispensable. La grosse différence entre less et cat, c'est que less affiche progressivement le contenu du fichier,

page par page. Ça vous laisse le temps de le lire dans la console. Comment ça marche ? Eh bien la commande est très simple : `less nomdufichier`.

```
$ less syslog
```

Ce qui est intéressant pour nous ici, c'est que la commande `less` a arrêté la lecture du fichier au bout de quelques lignes (la taille d'un écran de console). Cela vous laisse le temps de lire le début du fichier.

3.3.3 Les raccourcis basiques indispensables

Commençons par les quelques raccourcis clavier les plus indispensables, à connaître absolument.

- Espace : affiche la suite du fichier. La touche Espace fait défiler le fichier vers le bas d'un « écran » de console. C'est celle que j'utilise le plus souvent.
- Entrée : affiche la ligne suivante. Cela permet donc de faire défiler le fichier vers le bas ligne par ligne. Vous pouvez aussi utiliser la touche de la flèche vers le bas.
- d : affiche les onze lignes suivantes (soit une moitié d'écran). C'est un peu l'intermédiaire entre Espace (tout un écran) et Entrée (une seule ligne).
- b : retourne en arrière d'un écran. Vous pouvez aussi appuyer sur la touche Page Up.
- y : retourne d'une ligne en arrière. Vous pouvez aussi appuyer sur la touche de la flèche vers le haut.
- u : retourne en arrière d'une moitié d'écran (onze lignes).
- q : arrête la lecture du fichier. Cela met fin à la commande `less`.

3.3.4 afficher le début et la fin d'un fichier : head & tail

a. Afficher le début d'un fichier : head

La commande `head` (« tête » en anglais) affiche seulement les premières lignes du fichier. Elle ne permet pas de se déplacer dans le fichier comme `less`, mais juste de récupérer les premières lignes.

```
$ head syslog
```

Le paramètre `-n`, suivi d'un nombre, permet d'afficher le nombre de lignes que vous voulez. Par exemple, si vous ne voulez que les trois premières lignes, tapez :

```
$ head -n 3 syslog
```

b. Afficher la fin du fichier : tail

Très intéressante aussi (voire même plus), la commande `tail` vous renvoie la fin du fichier, donc les dernières lignes.

`$ tail syslog`

On peut là encore utiliser -n suivi d'un nombre pour afficher les 5 dernières lignes :

`$ tail -n 5 syslog`

Il y a un autre paramètre à côté duquel vous ne pouvez pas passer : -f (f pour *follow*, « suivre » en anglais).

Ce paramètre magique ordonne à tail de « suivre » la fin du fichier au fur et à mesure de son évolution. C'est extrêmement utile pour suivre un fichier de log qui évolue souvent. Vous pouvez tester sur syslog par exemple :

`$ tail -f syslog`

Le problème de syslog c'est qu'il n'évolue pas forcément toutes les secondes. Mais si vous êtes patients et que vous regardez votre console, vous devriez le voir écrire de nouvelles lignes sous vos yeux au bout d'un moment.

Faites Ctrl C (Ctrl et C en même temps) pour arrêter la commande tail.

CHAPITRE 4

ÉDITEURS DE TEXTE

4.1. Nano, l'éditeur de texte du débutant

4.1.1 Nano est un éditeur de texte, pas un traitement de texte !

Un éditeur de texte est un programme qui permet de modifier des fichiers de texte brut, sans mise en forme (gras, italique, souligné. . .). Sous Windows, on dispose d'un éditeur de texte très basique : le Bloc-Notes. Sous Linux, on a le choix entre Nano, Vim, Emacs et bien d'autres, sachant qu'au moins un de ceux-là est installé par défaut sur la plupart des distributions. Un traitement de texte est fait pour rédiger des documents mis en forme. Sous Windows, Word est le plus célèbre traitement de texte. sous Linux, on possède l'équivalent : Open Office Writer.

4.1.2 Découverte de Nano

Le nom complet de Nano est « GNU nano », en référence au projet GNU. Pour démarrer le logiciel, il vous suffit simplement de taper nano dans la console :

```
nano
```

L'éditeur Nano s'ouvre immédiatement



Dès lors, vous pouvez commencer à taper du texte. C'est aussi simple que cela ! Ne riez pas, je précise qu'il « suffit de taper du texte » car ce n'est pas aussi simple sous d'autres éditeurs, comme Vim par exemple. En bas de votre écran, vous pouvez voir un espace d'aide. Que signifie-t-il exactement ? Il s'agit d'un aide-mémoire pour vous rappeler à tout moment les commandes principales que vous pouvez lancer sous Nano. Le symbole ^ signifie Ctrl (la touche Contrôle de votre clavier). Ainsi, pour quitter Nano, il suffit de taper Ctrl + X.

Voici les raccourcis les plus importants :

- Ctrl + G : afficher l'aide ;
- Ctrl + K : couper la ligne de texte (et la mettre dans le presse-papier) ;
- Ctrl + U : coller la ligne de texte que vous venez de couper ;
- Ctrl + C : afficher à quel endroit du fichier votre curseur est positionné (numéro de ligne. . .) ;
- Ctrl + W : rechercher dans le fichier ;
- Ctrl + O : enregistrer le fichier (écrire) ;
- Ctrl + X : quitter Nano.

La recherche La combinaison de touches Ctrl + W lance une recherche dans le fichier

```
GNU nano 2.0.6          Nouvel espace          Modifié
Salut les Zéros, ça va ?
Je teste nano ! Un deux... un deux...

Recherche:
^G Aide      ^G Prem. lig.^G Remplacer ^G Début para.^G Resp.cass.^G Exp. ratio.
^G Annuler   ^G Dern. Lig.^G Aller lig.^G Fin para. ^G ->Arrière^G Précédente
```

Il vous suffit d'écrire le mot que vous recherche

```
GNU nano 2.0.6          Nouvel espace          Modifié
Salut les Zéros, ça va ?
Je teste nano ! Un deux... un deux...

Recherche: deux
^G Aide      ^G Prem. lig.^G Remplacer ^G Début para.^G Resp.cass.^G Exp. ratio.
^G Annuler   ^G Dern. Lig.^G Aller lig.^G Fin para. ^G ->Arrière^G Précédente
```

. . . puis de taper Entrée. Le curseur est automatiquement positionné à la première occurrence trouvée. Si le curseur est à la fin, la recherche recommence du début.

```
GNU nano 2.0.6          Nouvel espace          Modifié
Salut les Zéros, ça va ?
Je teste nano ! Un deux... un deux...

[ La recherche a fait le tour ]
^G Aide      ^G Ecrire   ^G Lire fich.^G Page préc.^G Couper    ^G Pos. cur.
^G Quitter   ^G Justifier ^G Chercher  ^G Page suiv.^G Coller    ^G Orthograp.
```

Enregistrer et quitter Pour enregistrer à tout moment, faites Ctrl + O. Si vous essayez de quitter (Ctrl + X) sans enregistrer auparavant, un message vous demandera si vous voulez sauvegarder. Si vous appuyez sur la touche o, vous passerez en mode enregistrement. Si vous appuyez sur la touche n, Nano quittera sans enregistrer. Si vous utilisez la combinaison Ctrl + C, vous annulerez votre demande de sortie de Nano et ne quitterez donc pas le logiciel. En appuyant sur o, vous vous retrouvez en mode enregistrement. Tapez juste le nom du fichier que vous voulez créer puis pressez Entrée. Après ça, Nano sera fermé et vous retrouverez votre bonne vieille ligne de commandes

4.1.3 Les paramètres de la commande Nano

Lorsque vous appelez Nano dans la ligne de commandes, vous pouvez spécifier plusieurs paramètres. Le plus courant est d'indiquer en paramètre le nom du fichier qu'on veut ouvrir. Ainsi

```
$ nano salut.txt
```

. . . ouvrira le fichier salut.txt que l'on vient de créer. Si le fichier n'existe pas, il sera automatiquement créé par Nano lors du premier enregistrement.

À part ça, la commande nano accepte de nombreux paramètres. Pour vous, j'en ai sélectionné trois qui me semblent faire partie des plus utiles.

- **-m** : autorise l'utilisation de la souris sous Nano. En console, oui, oui. Vous pouvez vous en servir pour cliquer avec votre souris sur la zone de texte où vous voulez placer votre curseur.

- **-i** : indentation automatique. L'alinéa (tabulations) de la ligne précédente sera respecté lorsque vous irez à la ligne. Très utile lorsque vous éditez un fichier de code source.

- **-A** : active le retour intelligent au début de la ligne. Normalement, lorsque vous appuyez sur la touche Origine (aussi connue sous le nom de Home) située à côté de la touche Fin, le curseur se repositionne au tout début de la ligne. Avec cette commande, il se positionnera après les alinéas. Comme -i, il s'agit d'une option utile avant tout pour les programmeurs. Si je veux lancer Nano avec toutes ces options à la fois, je peux donc écrire :

```
$ nano -miA salut.txt
```

CHAPITRE 5

LES UTILISATEURS ET LES DROITS

Linux est un système multi-utilisateur. Cela signifie que plusieurs personnes peuvent Travailler simultanément sur le même OS, en s'y connectant à distance notamment. Puisque plusieurs utilisateurs peuvent être connectés à Linux en même temps, celui-ci doit avoir une excellente organisation dès le départ. Ainsi chaque personne a **son** propre compte utilisateur, et il existe un ensemble de règles qui disent qui a le droit de faire quoi.

5.1. sudo : exécuter une commande en root

Lorsque vous avez installé Ubuntu, on vous a demandé le nom du compte utilisateur que vous vouliez créer. Dans la plupart des distributions Linux on vous proposera de créer un compte utilisateur avec des droits limités, c'est une mesure de sécurité. C'est une sécurité de ne pas avoir le droit de tout faire par défaut, car certaines commandes peuvent être dangereuses pour la stabilité et la sécurité de votre ordinateur. Avoir des droits limités, cela signifie aussi qu'on s'empêche par exemple d'exécuter la « commande de la mort qui tue » qu'on a vue dans le chapitre précédent (**rm -rf /***).

5.1.1 Devenir root un instant : sudo

On peut devenir root **temporairement** à l'aide de la commande **sudo**. Cette commande signifie « Faire en se substituant à l'utilisateur » : **S**ubstitute **U**ser **D**O. Écrivez donc **sudo** suivi de la commande que vous voulez exécuter, comme ceci :

```
sudo commande
```

On vous demandera normalement votre mot de passe (au moins la première fois) pour exécuter la commande. Ce mot de passe est le même que celui de votre compte utilisateur limité. Par exemple, vous pouvez exécuter un simple ls avec les droits root (vous ne risquez rien, rassurez-vous) :

```
mateo21@mateo21 - desktop :/ home$ sudo ls  
[ sudo ] password for mateo21 :
```

5.1.2 Devenir root et le rester : sudo su

Si vous tapez sudo su (tout court), vous passerez root indéfiniment.

```
mateo21@mateo21 - desktop :/ home$ sudo su  
[ sudo ] password for mateo21 :  
root@mateo21 - desktop :/ home #
```

5.2. Gestion des utilisateurs : adduser

Maintenant que vous savez passer root (temporairement ou indéfiniment), nous allons pouvoir découvrir des commandes qui sont réservées à root. adduser et deluser sont de celles-là. Si vous essayez de les appeler avec votre utilisateur normal, on vous dira que vous n'avez pas le droit de les utiliser. Seul root peut gérer les utilisateurs.

5.2.1 Ajouter un utilisateur : adduser

La commande adduser permet d'ajouter un utilisateur. Vous devez au minimum fournir un paramètre : le nom de l'utilisateur à créer. Par exemple, pour créer un compte pour Patrick :

```
root@mateo21 - desktop :/ home # adduser patrick
```

Le répertoire personnel de patrick est automatiquement créé (**/home/patrick**) et son compte est préconfiguré. On vous demande ensuite de taper son mot de passe : compte est préconfiguré. On vous demande ensuite de taper son mot de passe :

```
Entrez le nouveau mot de passe UNIX :  
Retapez le nouveau mot de passe UNIX :  
passwd : le mot de passe a été mis à jour avec succ ès
```

Tapez le mot de passe de patrick puis faites **Entrée**. Retapez-le pour valider. Encore une fois, si vous ne voyez pas d'étoiles * quand vous tapez le mot de passe, c'est normal ; c'est une sécurité pour qu'on ne puisse pas compter le nombre de caractères derrière votre épaule. On vous propose ensuite de rentrer quelques informations personnelles sur patrick, comme son nom, son numéro de téléphone. . . Si vous voulez le faire, faites-le, mais sinon sachez que vous pouvez taper **Entrée** sans rien écrire. À la fin, on vous demande de confirmer par un « o » (oui) que tout est bon. Tapez **Entrée** et ça y est, le compte de patrick est créé !

5.2.2 changer le mot de passe : passwd

S'il était nécessaire de changer le mot de passe de patrick par la suite, utilisez la commande **passwd** en indiquant en paramètre le nom du compte à modifier.

```
root@mateo21 - desktop :/ home # passwd patrick  
Entrez le nouveau mot de passe UNIX :  
Retapez le nouveau mot de passe UNIX :  
passwd : le mot de passe a été mis à jour avec succ ès
```

Attention ! Si vous appelez passwd sans préciser de compte en paramètre, c'est le mot de passe de root que vous changerez !

5.2.3 supprimer un compte : deluser

Patrick vous ennuie ? patrick est parti ? Si son compte n'est plus nécessaire (ou que vous voulez vous venger) vous pouvez le supprimer avec **deluser**.

```
deluser patrick
```

Aucune confirmation ne vous sera demandée !

Ne supprimez en aucun cas votre compte utilisateur ! Par exemple, je ne dois surtout pas supprimer le compte mateo21. En effet, si je le fais, il n'y aura plus que root sur la machine et Ubuntu interdit de se logger en root. Par conséquent, au prochain démarrage de la machine vous ne pourrez pas vous connecter et vous serez complètement coincés ! Toutefois, cette commande seule ne supprime pas le répertoire personnel de patrick. Si vous voulez supprimer aussi son home et tous ses fichiers personnels, utilisez le paramètre `--remove-home` :

```
deluser --remove-home patrick
```

5.3. Modifier les droits d'accès : chmod

5.3.1 Le fonctionnement des droits

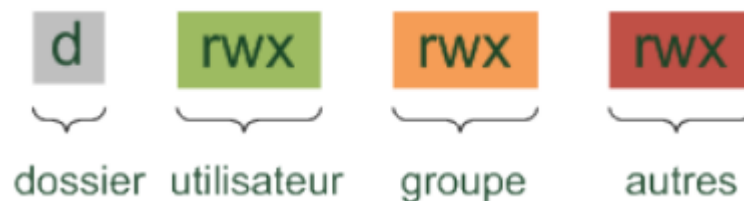
Chaque fichier et chaque dossier possède une liste de droits. C'est une liste qui indique qui a le droit de voir le fichier, de le modifier et de l'exécuter. Vous avez déjà vu des listes de droits, oui oui ! Lorsque vous faites un `ls -l`, il s'agit de la première colonne :

```
mateo21@mateo21-desktop:~$ ls -l
total 40
drwxr-xr-x 2 mateo21 mateo21 4096 2007-11-13 21:53 Desktop
drwxr-xr-x 2 mateo21 mateo21 4096 2007-11-13 13:46 Documents
lrwxrwxrwx 1 mateo21 mateo21   26 2007-09-19 18:31 Examples -> /
usr/share/example-content
drwxr-xr-x 2 mateo21 mateo21 4096 2007-09-25 20:28 images
drwxr-xr-x 2 mateo21 mateo21 4096 2007-10-19 01:21 Images
drwxr-xr-x 3 mateo21 mateo21 4096 2007-09-25 11:11 log
drwxr-xr-x 2 mateo21 mateo21 4096 2007-10-19 01:21 Modèles
drwxr-xr-x 2 mateo21 mateo21 4096 2007-10-19 01:21 Musique
drwxr-xr-x 2 mateo21 mateo21 4096 2007-10-19 01:21 Public
-rw-r--r-- 1 mateo21 mateo21    0 2007-11-15 23:14 rapport.txt
drwxr-xr-x 3 mateo21 mateo21 4096 2007-09-19 19:51 tutos
drwxr-xr-x 2 mateo21 mateo21 4096 2007-10-19 01:21 Vidéos
```

Vous voyez tous ces **d**, **r**, **w** et **x** au début ? Ce sont ce qu'on appelle les droits d'accès du fichier ou dossier. On peut voir cinq lettres différentes. Voici leur signification :

- **d** (Directory) : indique si l'élément est un dossier ;
- **l** (Link) : indique si l'élément est un lien (raccourci) ;
- **r** (Read) : indique si on peut lire l'élément ;
- **w** (Write) : indique si on peut modifier l'élément ;
- **x** (eXecute) : si c'est un fichier, « **x** » indique qu'on peut l'exécuter. Ce n'est utile que pour les fichiers exécutables (programmes et scripts). Si c'est un dossier, « **x** » indique qu'on peut le « traverser », c'est-à-dire qu'on peut voir les sous-dossiers qu'il contient si on a le droit de lecture dessus. Si la lettre apparaît, c'est que le droit existe. S'il y a un tiret à la place, c'est qu'il n'y a aucun droit.

Les droits sont découpés en fonction des utilisateurs



Le premier élément **d** mis à part, on constate que **r**, **w** et **x** sont répétés trois fois en fonction des utilisateurs :

- le premier triplet **rwx** indique les droits que possède le **propriétaire** du fichier sur ce dernier ;
- le second triplet **rwx** indique les droits que possèdent les autres membres du **groupe** sur ce fichier ;
- enfin, le dernier triplet **rwx** indique les droits que possèdent tous les **autres** utilisateurs de la machine sur le fichier. Prenons un cas concret, le fichier **rapport.txt** :

```
mateo21@mateo21-desktop:~$ ls -l rapport.txt
-rw-r--r-- 1 mateo21 mateo21 0 2007-11-15 23:14 rapport.txt
```

Ses droits sont : **-rw-r--r--**

- **-** : le premier tiret indique qu'il ne s'agit pas d'un dossier. S'il y avait eu un **d** à la place, cela aurait indiqué qu'il s'agissait d'un dossier.
- **rw-** : indique que le propriétaire du fichier, mateo21 en l'occurrence, peut lire et modifier (et donc supprimer) le fichier. En revanche, il ne peut pas l'exécuter car il n'a pas de **x** à la fin. Je rappelle que quiconque peut modifier un fichier a aussi le droit de le supprimer.
- **r--** : tous les utilisateurs qui font partie du groupe mateo21 mais qui ne sont pas mateo21 peuvent seulement lire le fichier. Ils ne peuvent ni le modifier, ni l'exécuter. Avoir un nom de groupe identique au nom d'utilisateur peut embrouiller : si vous êtes aussi bien organisés que sur mon premier schéma, on parlera plutôt du groupe famille.

– **r-** : tous les autres (ceux qui ne font pas partie du groupe mateo21) peuvent seulement lire le fichier.

En résumé, ces droits nous apprennent que l'élément est un fichier, que mateo21 peut le lire et le modifier et que tous les autres utilisateurs peuvent seulement le lire. Souvenez-vous d'une chose : root a TOUS les droits. Il peut tout faire : lire, modifier, exécuter n'importe quel fichier.

5.3.2 Modifier les droits d'accès : **chmod**

Une précision importante pour commencer : contrairement aux commandes précédentes, vous n'avez pas besoin d'être root pour utiliser **chmod**. Vous devez juste être propriétaires du fichier dont vous voulez modifier les droits d'accès. **chmod** est un petit peu délicat à utiliser. En effet, on peut attribuer les droits sur un fichier / dossier via plusieurs méthodes différentes, la plus courante étant celle des chiffres

5.3.2.1 Attribuer des droits avec des chiffres (chmod absolu)

Droit	Chiffre
r	4
w	2
x	1

Si vous voulez combiner ces droits, il va falloir additionner les chiffres correspondants. Ainsi, pour attribuer le droit de lecture et de modification, il faut additionner 4+2, ce qui donne 6. Le chiffre 6 signifie donc « Droit de lecture et d'écriture ». Voici la liste des droits possibles et la valeur correspondante :

Droits	Chiffre	Calcul
—	0	0 + 0 + 0
r-	4	4 + 0 + 0
-w-	2	0 + 2 + 0
-x	1	0 + 0 + 1
rw-	6	4 + 2 + 0
-wx	3	0 + 2 + 1
r-x	5	4 + 0 + 1
rwX	7	4 + 2 + 1

Avec ça, on peut calculer la valeur d'un triplet de droits. Il faut faire le même calcul pour les droits que l'on veut attribuer au propriétaire, au groupe et aux autres. Par exemple, « 640 » indique les droits du propriétaire, du groupe et des autres (dans l'ordre).

- 6 : droit de lecture et d'écriture pour le propriétaire.
- 4 : droit de lecture pour le groupe.
- 0 : aucun droit pour les autres.

Le droit maximal que l'on puisse donner à tout le monde est 777 : droit de lecture, d'écriture et d'exécution pour le propriétaire, pour son groupe et pour tous les autres. Bref, avec un tel droit tout le monde peut tout faire sur ce fichier. Au contraire, avec un droit de 000, personne ne peut rien faire. . . à part root, bien sûr.

Pour changer les droits sur le fichier rapport.txt, et être le seul autorisé à le lire et l'éditer, je dois exécuter cette commande :

```
chmod 600 rapport .txt
```

Un petit ls -l pour voir le résultat :

```
mateo21@mateo21 - desktop :~$ ls -l rapport .txt
-rw ----- 1 mateo21 mateo21 0 2007 -11 -15 23:14 rapport .txt
```

5.3.2.2 Attribuer des droits avec des lettres (chmod relatif)

Il existe un autre moyen de modifier les droits d'un fichier. Il revient un peu au même mais permet parfois de paramétrer plus finement, droit par droit. Dans ce mode, il faut savoir que :

- **u** = user (propriétaire) ;
- **g** = group (groupe) ;
- **o** = other (autres).
- **+** signifie : « Ajouter le droit » ;
- **-** signifie : « Supprimer le droit » ;
- **=** signifie : « Affecter le droit ».

Maintenant que vous savez cela, vous pouvez écrire :

```
chmod g+w rapport .txt
```

Signification : « Ajouter le droit d'écriture au groupe ».

```
chmod o-r rapport .txt
```

Signification : « Enlever le droit de lecture aux autres ».

```
chmod u+rx rapport .txt
```

Signification : « Ajouter les droits de lecture et d'exécution au propriétaire ».

```
chmod g+w,o-w rapport .txt
```

Signification : « Ajouter le droit d'écriture au groupe et l'enlever aux autres ».

```
chmod go -r rapport .txt
```

Signification : « Enlever le droit de lecture au groupe et aux autres ».

```
chmod +x rapport .txt
```

Signification : « Ajouter le droit d'exécution à tout le monde ».

```
chmod u=rwx,g=r,o=- rapport .txt
```

Signification : « Affecter tous les droits au propriétaire, juste la lecture au groupe, rien aux autres »

CHAPITRE 6

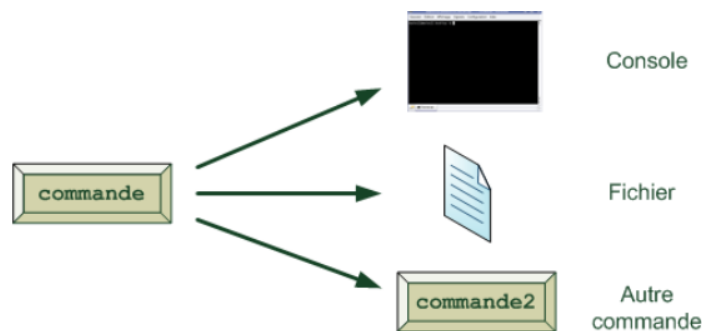
LES FLUX DE REDIRECTION

Vous devriez maintenant avoir l'habitude d'un certain nombre de commandes que propose la console de Linux. Le fonctionnement est toujours le même :

1. vous tapez la commande (par exemple `ls`) ;
2. le résultat s'affiche dans la console.

Ce que vous ne savez pas encore, c'est qu'il est **possible de rediriger ce résultat**. Au lieu que celui-ci s'affiche dans la console, vous allez pouvoir l'envoyer ailleurs : dans un fichier ou en entrée d'une autre commande. Ainsi, le résultat d'une commande peut en déclencher une autre ! **Comment ?** À l'aide de petits symboles spéciaux, appelés **flux de redirection**.

Le principe peut être résumé dans le schéma de la figure suivante



Jusqu'ici, nous n'avons donc exploité que la première possibilité (celle par défaut) : afficher le résultat dans la console. Il nous reste donc bien d'autres techniques à découvrir !

6.1 Rediriger le résultat dans un fichier : > et >>

La manipulation la plus simple que nous allons voir va nous permettre d'**écrire le** résultat d'une commande dans un fichier, au lieu de l'afficher dans la console.

6.1.1. rediriger dans un nouveau fichier : >

Ce symbole > permet de rediriger le résultat de la commande dans le fichier de votre choix.

```
$ ls -l > resultat.txt
```

```
$ cat resultat.txt
```

Si le fichier n'existe pas, il sera créé. S'il existe, son contenu sera écrasé, même si la commande tapée est incorrecte. Le shell commence d'abord par créer le fichier puis

exécute ensuite la commande. C'est un aspect important des redirections : les redirections sont interprétées de la droite vers la gauche, et les redirections sont mises en place AVANT l'exécution des commandes : il faut bien créer le fichier avant de pouvoir y écrire. D'où le fait que même si la commande est fausse, le fichier est créé ou écrasé...

6.1.2 rediriger à la fin d'un fichier : >>

Pour rajouter des données à la suite du fichier, donc sans l'écraser, on utilise la double redirection >>. Le résultat de la commande est ajouté à la fin du fichier.

```
$ ls -l > resultat.txt
```

```
$ date >> resultat.txt
```

```
$ cat resultat.txt
```

Avantage : vous ne risquez pas d'écraser le fichier s'il existe déjà. Si le fichier n'existe pas, il sera créé automatiquement.

6.1.3 rediriger les erreurs : 2>, 2>> et 2>&1

Il faut savoir que toutes les commandes produisent deux flux de données différents :

- **la sortie standard** : pour tous les messages (sauf les erreurs) ;
- **la sortie d'erreurs** : pour toutes les erreurs.

Prenons un exemple concret pour voir comment ça se passe. Supposons que vous fassiez un cat du fichier **notes.csv** pour afficher son contenu. Il y a deux possibilités :

- **si tout va bien**, le résultat (le contenu du fichier) s'affiche sur la sortie standard ;
- **s'il y a une erreur**, celle-ci s'affiche dans la sortie d'erreurs.

Par défaut, tout s'affiche dans la console : la sortie standard comme la sortie d'erreurs. Cela explique pourquoi vous ne faisiez pas la différence entre ces deux sorties jusqu'ici elles avaient l'air identiques.

a) Rediriger les erreurs dans un fichier à part

On pourrait souhaiter enregistrer les erreurs dans un fichier à part pour ne pas les oublier et pour pouvoir les analyser ensuite.

Pour cela, on utilise l'opérateur **2>**. Vous avez bien lu : c'est le chiffre 2 collé au chevron que nous avons utilisé tout à l'heure. Exemple. Exécuter les commandes **ls** et **date** (chainer les 2 commandes)

```
$ ls ; date
```

Ensuite, Exécuter les commandes en faisant exprès de mal saisir la commande date :

```
ls; dates
```

```
$ ls ; dates
```

La commande dates produit une erreur qui s'affiche à l'écran. On va maintenant rediriger la sortie d'erreur dans un fichier

```
$ ls > resultats.txt ; dates 2> erreurs.txt
```

Ceci redirige le résultat de la commande (sauf les erreurs) dans le fichier resultats.txt. C'est la sortie standard, et redirige les erreurs éventuelles dans le fichier erreurs.txt

b) Fusionner les sorties

Parfois, on n'a pas envie de séparer les informations dans deux fichiers différents. Heureusement, il est possible de fusionner les sorties dans un seul et même fichier. Comment ? Il faut utiliser le code suivant : **2>&1**. Cela a pour effet de rediriger toute la sortie d'erreurs dans la sortie standard. Traduction pour l'ordinateur : « envoie les erreurs au même endroit que le reste ».

```
$ lss -l > result.txt 2>&1
```

6.2 chaîner les commandes : |

Chaîner des commandes ? Cela signifie connecter la sortie d'une commande à l'entrée d'une autre commande



En gros, **tout ce qui sort de la commande1 est immédiatement envoyé à la commande2**. Et vous pouvez chaîner des commandes comme cela indéfiniment !

6.2.1 le pipe |

Le symbole | (Alt Gr + 6) appelé **pipe** | (prononcez « païpe », comme un bon Anglais) permet de chaîner des commandes.

Créer un fichier *noms.txt* contenant les noms suivants :

Fabrice ; Mathieu ; Sophie ; Mélanie ; Corentin ; Albert ; Mélanie ; Benoît,

On va afficher ces noms triés dans l'ordre alphabétiques

```
$ cat nom.txt | sort
```

On peut même aller plus loin et écrire cette liste triée dans un fichier :

```
$ cat nom.txt | sort > noms_tries .txt
```

CHAPITRE 7

AUTOMATISATION DES TACHES

7.1 Exécuter un programme a une heure différée

Nous savons lancer une commande pour qu'elle s'exécute tout de suite. Il est cependant aussi possible de « retarder » son lancement. Linux vous propose toute une série d'outils qui vous permettent de programmer à l'avance l'exécution d'une tâche, comme par exemple la **crontab** que nous allons étudier. Tous les outils que nous allons découvrir dans ce chapitre feront en outre appel à la notion de date. Nous allons donc dans un premier temps nous intéresser au formatage de la date. La commande `date` permet d'afficher la date

7.1.1 Personnaliser l'affichage de la date

Pour spécifier un affichage personnalisé, vous devez utiliser un symbole `+` suivi d'une série de symboles qui indiquent l'information que vous désirez. Je vous recommande de mettre le tout entre guillemets. Prenons quelques exemples pour bien comprendre :

```
$ date "+%H"
```

Le `+%H` est le **format de date**. `%H` signifie « le numéro de l'heure actuelle ».

Essayons autre chose d'un peu plus compliqué :

```
$ date "+%H:%M:%S"
```

Ici, j'ai rajouté les minutes (`%M`) et les secondes (`%S`). J'ai séparé les nombres par des deux-points, mais j'aurais très bien pu mettre autre chose à la place :

```
$ date "+% Hh%Mm%Ss"
```

Seule la lettre qui suit le `%` est interprétée. Mes lettres « `h` », « `m` » et « `s` » sont donc simplement affichées. Exécuter la commande suivante

```
$ date "+ Bienvenue en %Y"
```

7.1.2 Modifier la date

La commande `date` permet aussi de changer la date. Attention, il faudra être `root` pour faire cela Il faut préciser les informations sous la forme suivante : `MMDDhhmmYYYY`.

Les lettres signifient :

- `MM` : mois ;
- `DD` : jour ;
- `hh` : heure ;
- `mm` : minutes ;

– YYYY : année.

Notez qu'il n'est pas obligatoire de préciser l'année. On peut donc écrire :

```
$ sudo date 11052306
```

La nouvelle date s'affiche automatiquement et elle est mise à jour sur le système.

Attention à bien respecter l'ordre des nombres : Mois - Jour - Heure - Minutes.

Et si je veux exécuter la commande demain à 14 h 17 et non pas aujourd'hui

```
$ at 14:17 tomorrow      # tomorrow signifie « demain ».
```

Et si je veux exécuter la commande le 15 novembre à 14 h 17 ?

```
$ at 14:17 11/15/24
```

Que fait l'exemple suivant ?

```
$ at 23:15
```

```
at> ls -l > liste.txt
```

```
at> touch fax.txt
```

```
at> ls -l >> liste.txt
```

7.2 Exécuter une commande après un certain délai

Il est possible d'exécuter une commande dans 5 minutes, 2 heures ou 3 jours sans avoir à écrire la date. Par exemple, pour exécuter la commande dans 5 minutes :

```
$ at now +5 minutes
```

ce qui signifie « Dans maintenant (*now*) + 5 minutes ». Les mots-clés utilisables sont les suivants :

- minutes ;
- hours (heures) ;
- days (jours) ;
- weeks (semaines) ;
- months (mois) ;
- years (années).

Un autre exemple :

```
$ at now +2 weeks
```

exécutera les commandes dans deux semaines.

7.3 lister et supprimer les *jobs* en attente : **atq** et **atrm**

Chaque fois qu'une commande est « enregistrée », at nous indique un numéro de job ainsi que l'heure à laquelle il sera exécuté.

Il est possible d'avoir la liste des jobs en attente avec la commande **atq** :

\$ atq

5 *Tue Nov 5 23:33:00 2024 a genie*

6 *Tue Nov 5 23:29:00 2024 a genie*

Si vous souhaitez supprimer le job n° 5, utilisez atrm :

\$ atrm 5

7.4 sleep : faire une pause

Le saviez-vous ? Vous pouvez enchaîner plusieurs commandes à la suite en les séparant par des points-virgules comme ceci :

\$ touch fichier .txt; rm fichier .txt

touch est d'abord exécuté, puis une fois qu'il a fini ce sera le tour de rm (qui supprimera le fichier que nous venons de créer). Parfois, enchaîner les commandes comme ceci est bien pratique. mais on a besoin de faire une pause entre les deux. C'est là qu'intervient sleep : cette commande permet de faire une pause.

\$ touch fichier .txt; sleep 10; rm fichier .txt

Cette fois, il va se passer les choses suivantes :

- fichier.txt est créé ;
- sleep fait une pause de 10 secondes ;
- rm supprime ensuite le fichier.

Par défaut, la pause est exprimée en secondes. Il est aussi possible d'utiliser d'autres symboles pour changer l'unité :

- m : minutes ;
- h : heures ;
- d : jours.

Pour faire une pause d'une minute :

\$ touch fichier .txt; sleep 1m; rm fichier .txt

L'intérêt de sleep ne vous paraîtra peut-être pas évident tout de suite, mais retenez que cette commande existe car il est parfois bien pratique de faire une pause, par exemple pour s'assurer que la première commande a bien eu le temps de se terminer.

Vous pouvez aussi remplacer les points-virgules par des &&, comme ceci :

\$ touch fichier .txt && sleep 10 && rm fichier .txt

Dans ce cas, les instructions ne s'enchaîneront que si elles se sont correctement exécutées. Par exemple, si touch renvoie une erreur pour une raison ou une autre, alors les commandes qui suivent (sleep, rm) ne seront pas exécutées.

7.5 Exécuter une commande régulièrement : crontab

La « crontab » constitue un incontournable sous Linux : cet outil nous permet de programmer l'exécution régulière d'un programme. Contrairement à `at` qui n'exécutera le programme qu'une seule fois, `crontab` permet de faire en sorte que l'exécution soit répétée : toutes les heures, toutes les minutes, tous les jours, tous les trois jours, etc.

7.5.1 La « crontab », qu'est-ce que c'est ?

`crontab` est en fait une commande qui permet de lire et de modifier un fichier appelé la « crontab ». Ce fichier contient la liste des programmes que vous souhaitez exécuter régulièrement, et à quelle heure vous souhaitez qu'ils soient exécutés. `crontab` permet donc de changer la liste des programmes régulièrement exécutés. C'est toutefois le programme `cron` qui se charge d'exécuter ces programmes aux heures demandées. Ne confondez donc pas `crontab` et `cron` : le premier permet de modifier la liste des programmes à exécuter, le second les exécute. Comment utilise-t-on `crontab` ? Il y a trois paramètres différents à connaître, pas plus :

- **-e** : modifier la crontab ;
- **-l** : afficher la crontab actuelle ;
- **-r** : supprimer votre crontab. Attention, la suppression est immédiate et sans confirmation. Commençons par afficher la crontab actuelle :

```
$ crontab -l  
no crontab for mateo21
```

Normalement, vous n'avez pas encore créé de crontab. Vous noterez qu'il y a une crontab par utilisateur.

```
$ sudo crontab -l  
no crontab for root
```

Avant de modifier notre crontab, nous devons modifier notre configuration (notre fichier `.bashrc`) pour demander à ce que Nano soit l'éditeur par défaut. En général, c'est le programme « vi » qui fait office d'éditeur par défaut. C'est un bon éditeur de texte, mais bien plus complexe que Nano

```
export EDITOR = nano
```

Vous pouvez aussi écrire la commande suivante :

```
$ echo " export EDITOR = nano " >> ~/.bashrc
```

Cela aura pour effet d'écrire cette ligne à la fin de votre fichier `.bashrc` situé dans votre répertoire personnel. Fermez ensuite votre console et rouvrez-la pour que cette nouvelle configuration soit bien prise en compte.

```
$ 20 00 * * * touch /home/genie/fichierCront.txt
```

Chaque ligne du fichier correspond à une commande que l'on veut voir exécutée régulièrement. voici un schéma qui résume la syntaxe d'une ligne.

X X X X X Commande



- m : minutes (0 - 59) ;
- h : heures (0 - 23) ;
- dom (*day of month*) : jour du mois (1 - 31) ;
- mon (*month*) : mois (1 - 12) ;
- dow (*day of week*) : jour de la semaine (0 - 6, 0 étant le dimanche) ;
- command : c'est la commande à exécuter.

Une ligne qui commence par un #, est un commentaire

En clair, vous devez d'abord indiquer à quel moment vous voulez que la commande soit exécutée, puis ensuite écrire à la fin la commande à exécuter. C'est un peu comme un tableau. Chaque champ est séparé par un espace. Chaque « X » sur mon schéma peut être remplacé soit par un nombre, soit par une étoile qui signifie « tous les nombres sont valables ». Bien comprendre la crontab n'est pas si simple, je vous propose donc de nous baser sur quelques exemples pour voir comment ça fonctionne. Imaginons que je veuille exécuter une commande tous les jours à 15 h 47. Je vais écrire ceci :

```
$ 47 15 * * * touch/home/genie/fichier.txt
```

Seules les deux premières valeurs sont précisées : les minutes et les heures. Chaque fois qu'il est 15 h 47, la commande indiquée à la fin sera exécutée.

J'ai écrit le chemin du fichier en entier, car vous ne pouvez pas être sûrs que le cron s'exécutera dans le répertoire que vous voulez. Il est donc toujours préférable d'écrire le chemin du fichier en absolu comme je l'ai fait ici : /home/genie/fichier.txt.

Désormais, fichier.txt sera créé dans mon répertoire personnel tous les jours à 15 h 47 (s'il n'existe pas déjà).

Revenez dans la crontab, nous allons voir d'autres exemples (tableau suivant).

Crontab	Signification
47 * * * * commande	Toutes les heures à 47 minutes exactement.> & Donc à 00 h 47, 01 h 47, 02 h 47, etc.

0 0 * * 1 commande	Tous les lundis à minuit (dans la nuit de dimanche à lundi).
0 4 1 * * commande	Tous les premiers du mois à 4 h du matin.
0 4 * 12 * commande	Tous les jours du mois de décembre à 4 h du matin.
0 * 4 12 * commande	Toutes les heures les 4 décembre.
* * * * * commande	Toutes les minutes

Est-il possible d'exécuter une commande plus fréquemment que toutes les minutes ?

Non, c'est impossible avec cron. La fréquence minimale, c'est toutes les minutes.

7.5.2 Les différentes notations possibles

Pour chaque champ, on a le droit à différentes notations :

- 5 (un nombre) : exécuté lorsque le champ prend la valeur 5 ;
- * : exécuté tout le temps (toutes les valeurs sont bonnes) ;
- 3,5,10 : exécuté lorsque le champ prend la valeur 3, 5 ou 10. Ne pas mettre d'espace après la virgule ;
- 3-7 : exécuté pour les valeurs 3 à 7 ;
- */3 : exécuté tous les multiples de 3 (par exemple à 0 h, 3 h, 6 h, 9 h. . .).

Voici, sur le tableau suivant, quelques exemples d'utilisation.

Crontab	Signification
30 5 1-15 * * commande	À 5 h 30 du matin du 1er au 15 de chaque mois.
0 0 * * 1,3,4 commande	À minuit le lundi, le mercredi et le jeudi.
0 */2 * * * commande	Toutes les 2 heures (00 h 00, 02 h 00, 04 h 00. . .)
*/10 * * * 1-5 commande	Toutes les 10 minutes du lundi au vendredi.

Comme vous le voyez, la crontab offre de très larges possibilités (pour peu que l'on ait compris comment elle fonctionne)

7.6 Automatisez vos tâches avec des scripts Bash

7.6.1 scripts shell

Le shell est le programme qui gère l'invite de commandes. C'est donc le programme qui attend que vous rentriez des commandes. C'est aussi le programme qui est capable par exemple de :

- se souvenir quelles étaient les dernières commandes tapées (vous remontez dans votre historique en appuyant sur la flèche « Haut ») ;
- autocompléter une commande ou un nom de fichier lorsque vous appuyez sur Tab

Voici les noms de quelques-uns des principaux shells qui existent.

- **sh** : *Bourne Shell*. L'ancêtre de tous les shells.
- **bash** : *Bourne Again Shell*. Une amélioration du *Bourne Shell*, disponible par défaut

sous Linux et Mac OS X.

- **ksh** : *Korn Shell*. Un shell puissant assez présent sur les Unix propriétaires, mais aussi disponible en version libre, compatible avec bash.

- **csch** : *C Shell*. Un shell utilisant une syntaxe proche du langage C.

- **tcsh** : *Tenex C Shell*. Amélioration du *C Shell*.

- **zsh** : *Z Shell*. Shell assez récent reprenant les meilleures idées de bash, ksh et tcsh

Le bash (*Bourne Again Shell*) est le shell par défaut de la plupart des distributions Linux mais aussi celui du terminal de Mac OS X.

7.6.2 Notre premier script

Nous allons commencer par écrire un premier script bash tout simple. Il ne sera pas révolutionnaire mais va nous permettre de voir les bases de la création d'un script et comment celui-ci s'exécute. Cela sera donc essentiel pour la suite.

a) Création du fichier

Commençons par créer un nouveau fichier pour notre script. Le plus simple est d'ouvrir nano en lui donnant le nom du nouveau fichier à créer :

```
$ nano essai.sh
```

Si `essai.sh` n'existe pas, il sera créé (ce qui sera le cas ici). La première chose à faire dans un script shell est d'indiquer. . . quel shell est utilisé. En ce qui nous concerne, nous souhaitons utiliser la syntaxe de bash, plus répandu sous Linux et plus complet que sh. Nous indiquons où se trouve le programme bash :

```
#!/bin/bash
```

Le `#!` est appelé le sha-bang. `/bin/bash` peut être remplacé par `/bin/sh` si vous souhaitez coder pour sh, `/bin/ksh` pour ksh, etc.

b) Exécution de commandes

Après le sha-bang, nous pouvons commencer à coder. Le principe est très simple : il vous suffit d'écrire les commandes que vous souhaitez exécuter. Ce sont les mêmes que celles que vous tapiez dans l'invite de commandes !

- `ls` : pour lister les fichiers du répertoire.

- `cd` : pour changer de répertoire.

- `mkdir` : pour créer un répertoire.

- `sort` : pour trier des mots.

- etc.

c) Les commentaires

Notez que vous pouvez aussi ajouter des commentaires dans votre script. Ce sont des lignes qui ne seront pas exécutées mais qui permettent d'expliquer ce que fait votre script. Tous les commentaires commencent par un #. Par exemple :

```
#!/ bin/ bash  
# Affichage de la liste des fichiers
```

Vous avez sûrement remarqué que la ligne du sha-bang commence aussi par un #. . . Oui, c'est un commentaire aussi, mais considérez que c'est un commentaire « spécial » qui a un sens. Il fait un peu exception.

7.6.3 Exécuter le script bash

a) Donner les droits d'exécution au script

Si vous faites un `ls -l` pour voir votre fichier qui vient d'être créé, vous obtenez ceci :

```
$ ls -l  
total 4  
-rw -r--r-- 1 mateo21 mateo21 17 2009 -03 -13 14:33 essai .sh
```

Ce qui nous intéresse ici, ce sont les droits sur le fichier : `-rw-r--r--`. on se rend compte que notre script peut être lu par tout le monde (r), écrit uniquement par nous (w), et n'est pas exécutable (pas de x). Or, pour exécuter un script, il faut que le fichier ait le droit « exécutable ». Le plus simple pour donner ce droit est d'écrire :

```
$ chmod +x essai .sh
```

Vous pouvez vérifier que le droit a bien été donné :

```
$ ls -l  
total 4  
-rwxr -xr -x 1 mateo21 mateo21 17 2009 -03 -13 14:33 essai .sh
```

b) Exécution du script

Le script s'exécute maintenant comme n'importe quel programme, en tapant « `./` » devant le nom du script

```
$ ./ essai .sh
```

c) Exécution de débogage

Plus tard, vous ferez probablement de gros scripts et risquerez de rencontrer des bugs. Il faut donc dès à présent que vous sachiez comment déboguer un script. Il faut l'exécuter comme ceci :

```
$ bash -x essai .sh
```

On appelle en fait directement le programme bash et on lui ajoute en paramètre un -x (pour lancer le mode débogage) ainsi que le nom de notre script à déboguer. Le shell affiche alors le détail de l'exécution de notre script, ce qui peut nous aider à retrouver la cause de nos erreurs :

```
$ bash -x essai .sh  
+ pwd  
/ home / mateo21 / scripts  
+ ls  
essai .sh
```


CHAPITRE 8

LA CONNEXION SECURISEE A DISTANCE AVEC SSH

8.1. Introduction

Nous allons découvrir comment se connecter à distance à une machine équipée de Linux. Jusqu'ici, vous avez utilisé Linux de la même façon que Windows : vous étiez **en face** de votre ordinateur. Vous étiez physiquement à côté de votre machine, vous avez par exemple appuyé sur le bouton « Power » pour l'allumer. Jusque-là, rien de nouveau. Pourtant, une des grandes forces de Linux est que l'on peut s'en servir même si l'on est à des centaines de kilomètres de la machine. Ce genre de manipulation est désormais possible grâce à l'internet, et cela se fait tous les jours. Les personnes qui s'emploient à gérer des machines Linux, souvent à distance, sont appelées **administrateurs système** (c'est un métier recherché !). Heureusement qu'il n'est pas nécessaire d'être présent physiquement à côté de la machine pour travailler dessus ! Vous imaginez, devoir se payer un billet aller-retour pour Tokyo juste parce que l'on a besoin d'installer un programme sur un serveur. . . Un **serveur** est un ordinateur qui reste allumé 24 h / 24, 7 j / 7. Cet ordinateur est semblable au vôtre (quoique souvent plus puissant et plus bruyant) : il possède un processeur, un ou plusieurs disques durs, etc. Le principe d'un serveur est de rester allumé et connecté à l'internet tout le temps. Il offre des services. Le PC qui se connecte au serveur est appelé le **client**.



Actuellement, votre petit PC chez vous n'est pas considéré comme un serveur. . . mais vous pouvez très facilement le transformer en serveur si vous le désirez, à condition d'installer les bons programmes et de les configurer correctement. Et de le laisser allumé aussi, parce qu'un serveur éteint, c'est un serveur qui ne sert à rien.

8.2 De Telnet à SSH

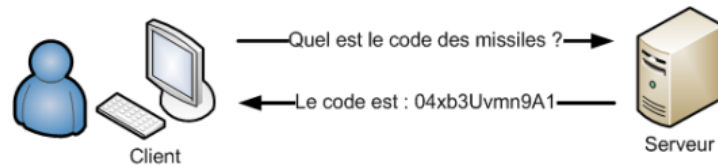
8.2.1 Les protocoles

Pour communiquer entre eux en réseau, deux ordinateurs doivent utiliser le même **protocole**.

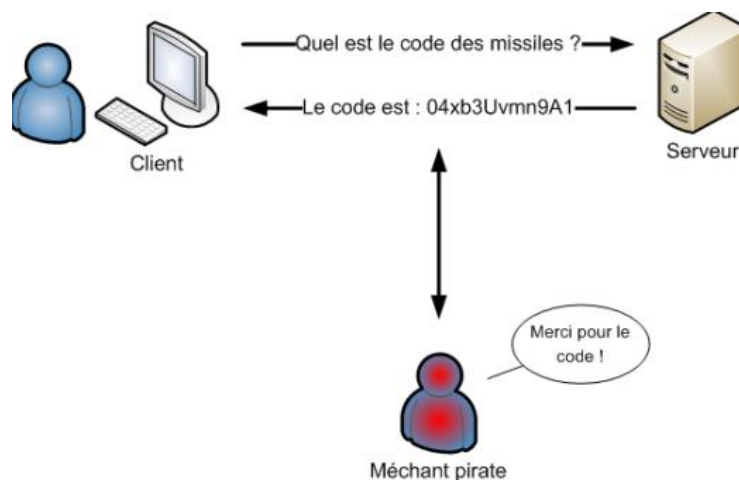
8.2.2 Le protocole Telnet : simple mais dangereux

Un protocole très simple, très basique, a été créé dans les années 80 : c'est **Telnet**. Il sert juste à échanger des messages simples d'une machine à une autre. En théorie

donc, on peut communiquer avec un serveur à l'aide du protocole Telnet. Le problème de ce protocole. . . c'est justement qu'il est trop simple : les données sont transférées en **clair** sur le réseau. Il n'y a **aucun cryptage**. Voici ce qui pourrait se passer. Imaginez qu'un PC militaire demande à un serveur de l'armée le code de lancement de missiles (nucléaires)



Après tout, il n'y a rien de choquant. Le message n'est envoyé qu'au client qui l'a demandé. Mais en fait, un pirate aurait la possibilité d'« **écouter** » ce qui se passe sur le réseau, et donc d'intercepter les données en chemin



8.2.3 Le protocole SSH : la solution pour sécuriser les données

Comme on ne peut pas complètement empêcher quelqu'un d'intercepter les données qui transitent sur l'internet, il faut trouver un moyen pour que le client et le serveur communiquent de manière sécurisée. Le cryptage sert précisément à ça : si le pirate récupère le mot de passe crypté, il ne peut rien en faire. **SSH** est un protocole assez complexe, mais vraiment intéressant

8.3 Se connecter avec SSH et PuTTY

8.3.1 Transformer sa machine en serveur

Cette étape vous concerne si vous voulez transformer votre PC en serveur. Par exemple, si vous voulez accéder à votre PC depuis un autre lieu, Il faut tout simplement installer le paquet openssh-server :

```
$ sudo apt -get install openssh - server
```

Ensuite, le programme de serveur SSH (appelé sshd) est lancé. Normalement, le serveur SSH sera lancé à chaque démarrage. Si ce n'est pas le cas, vous pouvez le lancer à tout moment avec la commande suivante :

```
$ sudo /etc/ init .d/ssh start
```

Et vous pouvez l'arrêter avec cette commande :

```
$ sudo /etc/ init .d/ssh stop
```

Logiquement, vous ne devriez pas avoir besoin de configurer quoi que ce soit, mais sachez au besoin que le fichier de configuration se trouve dans /etc/ssh/ssh_config.

Il faudra recharger SSH avec la commande `sudo /etc/init.d/ssh reload` pour que les changements soient pris en compte. Voilà : votre machine est désormais un serveur SSH ! Vous pouvez vous y connecter depuis n'importe quel ordinateur sous Linux ou sous Windows dans le monde (pour peu que vous ne soyez pas derrière un pare-feu).

8.3.1 Se connecter via SSH à partir d'une machine Linux

Toutes les machines équipées de Linux proposent la commande `ssh` qui permet de se connecter à distance à une autre machine. Ouvrez une console sur le second PC et utilisez la commande `ssh` comme ceci :

```
$ ssh login@ip
```

Il faut remplacer login par votre login (mateo21, dans mon cas) et ip par l'adresse IP de votre ordinateur.

Si vous vous connectez depuis chez un ami, il vous faut entrer l'IP internet de votre PC que vous pouvez obtenir en allant sur www.whatismyip.com par exemple. Si vous vous connectez depuis un autre PC chez vous (sur le même réseau local), il vous faut entrer l'IP locale que vous devriez voir en tapant la commande `ifconfig` (par exemple 192.168.0.3). Si vraiment vous n'avez ni ami sous Linux ni second PC dans la maison, vous pouvez simuler une connexion réseau en vous connectant depuis votre PC à votre PC. Utilisez pour cela l'IP 127.0.0.1 (ou le mot `localhost`), ça marche toujours.

Si vous devez changer l'adresse IP de votre machine, utilisez la commande suivante :

```
$ sudo ip addr add 192.168.2.9 dev eth1
```

Si je suis chez un ami et que l'IP internet de mon ordinateur est 87.112.13.165, je vais taper : `$ ssh mateo21@87.112.13.165`

Si, faute de mieux, vous voulez tester en vous connectant chez vous depuis chez vous, vous pouvez taper :

```
$ ssh mateo21@localhost
```

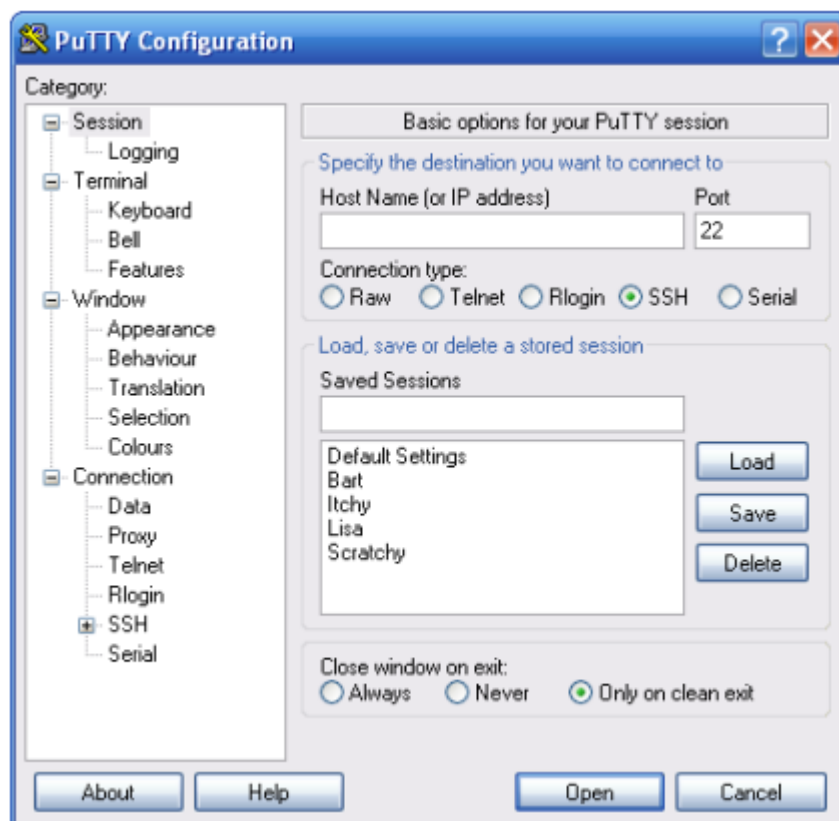
Si vous n'avez pas de réponse du serveur, vérifiez que vous ne vous êtes pas trompés d'IP. Vérifiez aussi que le port 22 n'est pas bloqué par un pare-feu, car c'est celui utilisé

par SSH par défaut. Si le serveur tourne sur un autre port, il faudra préciser le numéro de ce port comme ceci : `ssh mateo21@87.112.13.165 -p 12451` (si le serveur fonctionne sur le port 12451 au lieu du port 22). Pour vous déconnecter, tapez `logout` ou son équivalent : la combinaison de touches `Ctrl + D`.

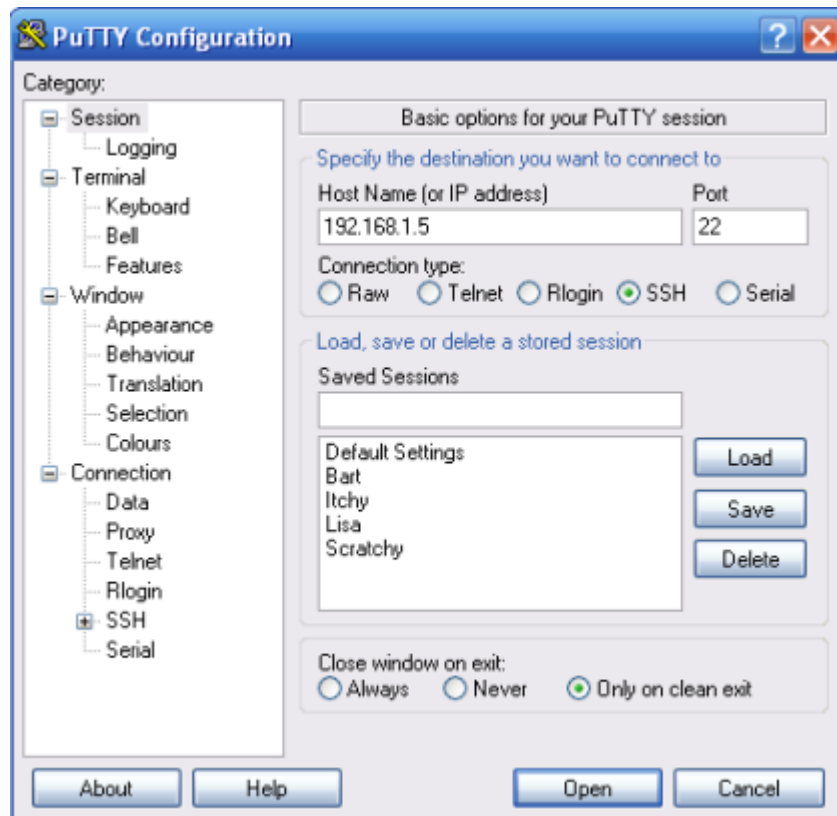
8.3.2 Se connecter via SSH à partir d'une machine Windows

Si vous voulez avoir accès à la console de votre machine Linux mais que vous n'avez pas d'autre machine Linux sous la main, pas de panique ! Il existe des programmes pour Windows faits pour cela. Le plus connu d'entre eux – celui que j'utilise personnellement – s'appelle **PuTTY**.

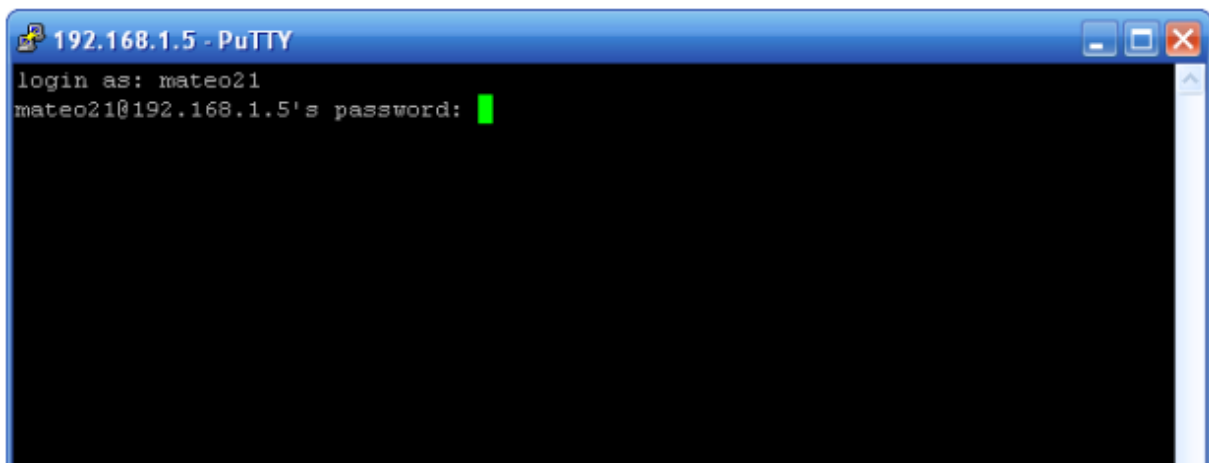
Une fois PuTTY installé, lancez PuTTY.



Il y a beaucoup de pages d'options, comme vous pouvez le voir au niveau de la section « Category » sur le côté. Pour le moment, pas de panique : vous avez juste besoin de remplir le champ en haut « Host Name (or IP address) ». Entrez-y l'adresse IP de votre ordinateur sous Linux. Dans mon cas, je vais entrer l'adresse IP de mon PC sous Linux situé sur mon réseau local (192.168.1.5). Vous pouvez changer le numéro du port si ce n'est pas 22, mais normalement c'est 22 par défaut. Ensuite, vous n'avez plus qu'à cliquer sur le bouton « Open » tout en bas pour lancer la connexion. Rien d'autre !



Le serveur vous demande alors le login et le mot de passe



Si tout est bon, vous devriez être connectés à votre machine! Pour vous déconnecter, tapez logout ou son équivalent, la combinaison de touches Ctrl + D.

CHAPITRE 9

TRANSFERER DES FICHIERS

9.1 Téléchargement de fichiers : wget

La commande wget permet de télécharger des fichiers directement depuis la console.

Il suffit d'indiquer l'adresse HTTP ou FTP d'un fichier à télécharger :

```
$ wget http://cdimage.debian.org/debian-cd/4.0_r5/i386/iso-cd/debian-4.0r5-i386-businesscard.iso
```

9.2 scp : copier des fichiers sur le réseau

Vous connaissez la commande cp ? Elle permet de copier des fichiers sur votre disque dur. Eh bien voici scp (*Secure CoPy*), qui permet de copier des fichiers d'un ordinateur à un autre à travers le réseau ! Le tout de manière sécurisée, bien sûr. scp s'utilise quasiment comme ssh. D'ailleurs ce n'est pas un hasard car scp s'appuie sur ssh pour fonctionner. Là où ssh sert à ouvrir une console à distance (un shell), scp est spécialement conçue pour copier des fichiers d'un ordinateur à un autre. On l'utilise comme ceci :

```
$ scp fichier_origine copie_destination
```

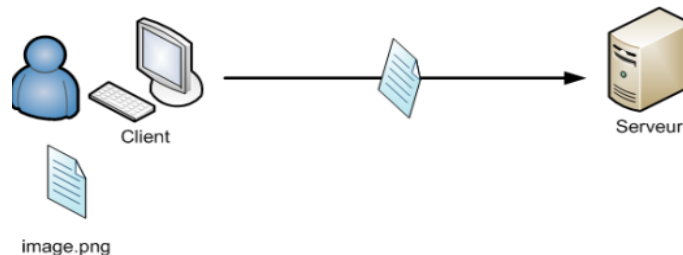
Le premier élément à indiquer est la position du fichier que l'on veut copier. Le second élément correspond au répertoire de destination où il doit être copié.

Chacun de ces éléments peut s'écrire sous la forme suivante : login@ip:nom_fichier. Le login et l'IP sont facultatifs. Si vous n'écrivez ni login ni IP, scp considérera que le fichier se trouve sur votre ordinateur.

9.3 Copier un fichier de votre ordinateur vers un autre

Prenons un cas concret pour que vous puissiez mieux voir comment ça s'utilise :

```
$ scp image.png mateo21@85.123.10.201:/home/mateo21/images/
```



Ici, je demande à copier le fichier image.png qui se trouve sur mon ordinateur vers un autre ordinateur dont l'IP est 85.123.10.201. Sur cet autre ordinateur, le fichier sera placé dans le dossier /home/mateo21/images/.

9.4 Copier un fichier d'un autre ordinateur vers le vôtre

On peut également faire le contraire (figure 24.2), c'est-à-dire récupérer un fichier qui se trouve sur un autre ordinateur et le placer chez nous :

```
$ scp mateo21@85.123.10.201: image.png copie_image_sur_mon_pc.png
```

Dans cet exemple, je copie le fichier image.png qui se trouve sur le serveur dont l'IP est 85.123.10.201 et place cette copie sur mon propre ordinateur sous le nom copie_image_sur_mon_pc.png. Si je veux, je peux aussi copier le fichier sans en changer le nom :

```
$ scp mateo21@85.123.10.201: image.png .
```

Notez le point à la fin. Il signifie « copier dans le répertoire dans lequel je me trouve ». Le fichier image.png sera donc placé sur mon ordinateur dans le dossier actuel.

9.5 ftp & sftp : transférer des fichiers

Le FTP (*File Transfer Protocol*) est un protocole permettant d'échanger des fichiers sur le réseau. Il est assez ancien (1985) et toujours utilisé à l'heure actuelle pour transférer des fichiers. On l'utilise en général dans deux cas.

- Pour télécharger un fichier depuis un serveur FTP public. En général, les navigateurs web font cela de manière automatique et transparente lorsque vous cliquez sur un lien de téléchargement. La connexion se fait alors en **mode anonyme**.

- Pour transférer des fichiers vers un serveur FTP privé (et éventuellement en télécharger aussi). Lorsque l'on prend un hébergement pour son site web, l'hébergeur nous donne en général des accès FTP pour aller y déposer les fichiers du site. La connexion se fait donc en **mode authentifié**.

9.5.1 Connexion à un serveur FTP

Essayons de nous connecter au serveur FTP de Debian, accessible à l'adresse suivante : ftp://ftp.debian.org

```
$ ftp ftp.debian.org
```

Le serveur FTP devrait répondre en vous demandant un login et un mot de passe. Pour les serveurs FTP publics, le login à utiliser est toujours anonymous (anonyme).

```
$ ftp ftp.debian.org
```

```
Connected to ftp.debian.org.
```

```
220 saens.debian.org FTP server (vsftpd)
```

```
Name (ftp.debian.org:mateo21): anonymous
```

```
331 Please specify the password.
```

```
Password :
```

Pour le mot de passe, peu importe ce que vous mettez, vous serez acceptés. :-) Vous devriez alors voir s'afficher un message de bienvenue se terminant par :

```
230 Login successful .Remote system type is UNIX .
```

```
Using binary mode to transfer files.
```

```
ftp >
```

Vous avez maintenant un **prompt** (il s'agit du terme anglais pour *invite de commandes*) ftp> qui vous permet de rentrer des commandes FTP.

9.5.2 Se déplacer au sein du serveur FTP

Vous savez quoi ? Bonne nouvelle : les commandes que vous pouvez utiliser sont pour la plupart les mêmes que celles que vous connaissez.

- ls : affiche le contenu du répertoire actuel ;
- pwd : affiche le chemin du répertoire actuel ;
- cd : change de répertoire.

Avec ces commandes, vous devriez déjà pouvoir vous balader sur le serveur FTP. Faites un ls pour voir :

9.6 Le transfert de fichiers

Si vous souhaitez récupérer un fichier ou en envoyer un, il y a deux commandes à connaître :

- put : envoie un fichier vers le serveur ;
- get : télécharge un fichier depuis le serveur.



Vous pouvez par exemple récupérer le fichier README en écrivant get README :

```
$ ftp > get README
```

Et voilà ! Le fichier se trouve maintenant sur votre ordinateur. Il a été téléchargé dans le dossier dans lequel vous vous trouviez sur votre ordinateur. Pour savoir dans quel dossier vous êtes **chez vous**, tapez !pwd Si vous voulez changer de dossier **chez vous**, utilisez !cd. Pour lister les fichiers **chez vous**, utilisez !ls. Bref, vous m'avez compris, il suffit de faire précéder les commandes d'un point d'exclamation pour qu'elles s'exécutent chez vous et non sur le serveur FTP. Pour quitter le serveur et vous déconnecter, vous avez le choix entre la bonne vieille combinaison de touches Ctrl + D

qui commande la fermeture de la session ou encore les commandes `bye`, `exit` et `quit`, qui sont analogues.

9.7. un FTP sécurisé : sftp

Le protocole FTP a un défaut : il n'est pas sécurisé ; les données ne sont pas cryptées. Quelqu'un ayant accès au réseau pourrait alors intercepter le contenu des fichiers que vous échangez ou encore votre mot de passe lors de la connexion.

Pour remédier à cela, on a inventé sftp, qui repose sur SSH pour sécuriser la connexion:

```
$ sftp login@ip
```

Par exemple :

```
$ sftp mateo21@lisa .simple -it.fr
```

On vous demandera alors votre mot de passe. Une fois que vous serez connectés, les commandes sont presque les mêmes que pour le FTP. Vous retrouverez notamment `get` et `put` pour échanger des fichiers. Sachez que les commandes sont globalement plus puissantes et pratiques en SFTP qu'en FTP. Méfiez-vous toutefois, certaines commandes changent ! Par exemple, pour supprimer un fichier, ce n'est plus `delete` mais à nouveau `rm` ! Lisez le manuel pour plus d'informations : `man sftp`.

Pour se connecter en SFTP, on utilise le même port que SSH (soit 22 par défaut). Si votre serveur SSH fonctionne sur un autre port, vous devrez le préciser comme ceci : `sftp -oPort=27401 mateo21@serveur`.

9.8 Synchroniser des fichiers pour une sauvegarde : rsync

`rsync` est un programme assez simple à utiliser et pourtant très puissant. Il permet d'effectuer une synchronisation entre deux répertoires, que ce soit sur le même PC ou entre deux ordinateurs reliés en réseau. `rsync` est le plus souvent utilisé pour effectuer des **sauvegardes incrémentielles**. Par exemple, imaginez que vous souhaitiez sauvegarder régulièrement les fichiers de votre home (répertoire personnel). Ainsi, si un jour un accident survient (perte du PC, par exemple), vous aurez toujours vos documents sauvegardés au chaud sur un serveur quelque part. D'accord, vous copiez tout votre home sur ce serveur. Oui mais voilà, votre home contient peut-être 10 Go de fichiers : entre vos vidéos, la musique, vos rapports au format texte et vos photos de vacances, ça va vite. Sauvegarder ces 10 Go une fois, d'accord. Mais la seconde fois, plutôt que de tout renvoyer, vous souhaiterez peut-être envoyer uniquement les fichiers qui ont été ajoutés ou modifiés. C'est là que `rsync` intervient. C'est une sorte de `scp` intelligent : il compare et analyse les différences entre deux dossiers puis copie

uniquement les changements. C'est ce que veut dire le mot « incrémentiel ». **rsync** peut être utilisé pour effectuer une sauvegarde entre deux dossiers sur le même ordinateur ou bien entre deux dossiers sur deux ordinateurs différents. En général, on l'utilise plutôt pour sauvegarder entre deux ordinateurs différents,

9.9 Sauvegarder dans un autre dossier du même ordinateur

Dans les exemples qui vont suivre, je vais supposer que vous souhaitez sauvegarder le dossier Images dans un dossier backups.

```
$ rsync -arv Images / backups /
```

Quelques explications concernant les paramètres :

- **-a** : conserve toutes les informations sur les fichiers, comme les droits (chmod), la date de modification, etc. ;
- **-r** : sauvegarde aussi tous les sous-dossiers qui se trouvent dans le dossier à sauvegarder ;
- **-v** : mode verbeux, affiche des informations détaillées sur la copie en cours.

Viennent ensuite le nom du dossier à sauvegarder et le répertoire de sauvegarde.

rsync analyse le contenu du répertoire de sauvegarde dans un premier temps. Comme celui-ci est vide, vous pouvez constater qu'il y a copié tous les fichiers. Maintenant, lancez la même commande une seconde fois. Comme vous pouvez le voir, cette fois aucun fichier n'a été envoyé ! En effet, rsync étant intelligent, il a détecté qu'il n'y avait aucun changement et donc qu'il n'y avait pas lieu de copier quoi que ce soit.

Testons un peu ce qui se passe si l'on ajoute un fichier :

```
$ touch Images / espagne2 .jpg$ rsync -arv Images / backups /
```

Le nouveau fichier espagne2.jpg a bien été copié ! ;-) Vous pouvez aussi essayer de modifier un fichier, vous verrez que rsync copie bien ceux qui ont été modifiés.

9.10 Sauvegarder sur un autre ordinateur

Intéressons-nous à la sauvegarde sur un autre ordinateur, parce que là c'est bien joli mais on se sentirait plus en sécurité si les fichiers étaient envoyés ailleurs, sur un autre ordinateur. L'avantage de rsync est qu'il peut copier les fichiers en employant plusieurs méthodes différentes. La plus couramment utilisée, que nous allons choisir ici, est de passer par SSH. Comme quoi, vous le voyez, SSH sert à sécuriser tous types de transferts.

```
$ rsync -arv -- delete -- backup --backup -dir =/ home / mateo21 / fichiers_supprimes / Images /  
mateo21@IP_du_serveur : mes_backups /
```

Si votre serveur SSH écoute sur un autre port que celui par défaut, il faudra rajouter -e "ssh -p port" :

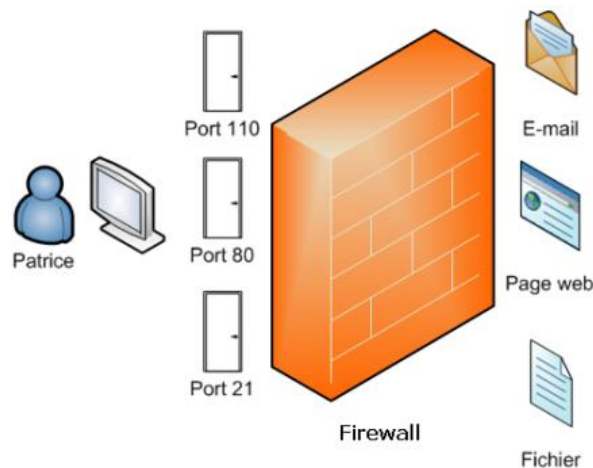
```
$ rsync -arv -- delete -- backup --backup -dir =/ home / mateo21 / fichiers_supprimes / Images/  
mateo21@IP_du_serveur : mes_backups / -e " ssh -p 12473"
```

CHAPITRE 10

ANALYSER LE RESEAU ET FILTRER LE TRAFIC AVEC UN PARE-FEU

10. 1 iptables : le pare-feu de référence

Le plus célèbre pare-feu utilisé sous Linux est iptables. Il permet d'établir un certain nombre de **règles** pour dire par quels ports *on peut* se connecter à votre ordinateur, mais aussi à quels ports vous *avez le droit* de vous connecter. On peut également filtrer par IP.



Par exemple, si je veux empêcher toute connexion FTP (parce que je considère que le FTP n'est pas sûr), je peux souhaiter bloquer le port 21 (utilisé par FTP). En général la technique ne consiste pas à bloquer certains ports mais plutôt à bloquer par défaut **tous** les ports et à en autoriser seulement quelques-uns. C'est avant tout une question de sécurité. Le but d'un pare-feu est d'empêcher que des programmes puissent communiquer sur le réseau sans votre accord. Aujourd'hui, même sous Windows (depuis Windows XP SP2), un pare-feu est intégré par défaut, tant le problème est important. Avoir un pare-feu ne vous prémunit pas contre les virus (bien que sous Linux, ils restent rares). En revanche, cela rend la tâche **particulièrement difficile** aux pirates qui voudraient accéder à votre machine. Chaque ordinateur possède plusieurs portes d'entrée possibles. Notre objectif est de bloquer par défaut toutes ces portes et d'autoriser seulement celles dont vous avez besoin, que vous considérez comme « sûres » et que vous utilisez. Par exemple, le port 80 utilisé pour le web est un port sûr que vous pouvez activer. Notez, et c'est important, qu'il y a des portes d'entrée et des portes de sortie sur votre ordinateur (ce ne sont pas nécessairement les mêmes). iptables est un programme extrêmement puissant, mais tout aussi complexe. iptables **s'utilise en « root »**. Pour manipuler iptables, vous devez impérativement être en « root ». Pour la suite des opérations, je vous recommande donc de passer en superutilisateur dès à présent :

```
$ sudo su
```

10.2. Afficher les règles : *iptables -L*

Avec *iptables -L* (attention, un « L » majuscule), vous pouvez afficher les règles qui régissent actuellement le pare-feu :

```
# iptables -L
Chain INPUT (policy ACCEPT)
target      prot opt source                destination

Chain FORWARD (policy ACCEPT)
target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target      prot opt source                destination
```

On repère trois sections :

- Chain INPUT : correspond aux règles manipulant le trafic entrant ;
- Chain FORWARD : correspond aux règles manipulant la redirection du trafic ;
- Chain OUTPUT : correspond aux règles manipulant le trafic sortant.

Nous ne verrons pas ici la section FORWARD. *iptables* permet de rediriger le trafic, mais c'est assez compliqué et ne nous intéresse pas ici. Nous aurons déjà suffisamment de quoi faire avec INPUT et OUTPUT. Actuellement, chez moi, les règles sont vides. Il y a trois tableaux mais qui ne contiennent aucune ligne. Par ailleurs, vous noterez à chaque fois les mots (policy ACCEPT) qui signifient que, par défaut, tout le trafic est accepté. Donc chez moi, pour le moment, le pare-feu est tout simplement inactif car il ne bloque rien ; mon ordinateur est une vraie passoire. Si vous avez déjà des règles inscrites dans votre pare-feu (ce qui ne devrait pas être votre cas !), sachez que vous pouvez les réinitialiser. Ne le faites que **si vous êtes certains de vouloir le faire**. En effet, sur un ordinateur partagé, peut-être quelqu'un a-t-il déjà configuré le pare-feu et il serait dommage de saboter tout son travail.

```
# iptables -F
```

10.3 Ajouter et supprimer des règles

Voici les principales commandes à connaître.

- **-A chain** : ajoute une règle en fin de liste pour la chain indiquée (INPUT ou OUTPUT, par exemple).
- **-D chain rulenum** : supprime la règle n° rulenum pour la chain indiquée.

- **-I chain rulenum** : insère une règle au milieu de la liste à la position indiquée par rulenum. Si vous n'indiquez pas de position rulenum, la règle sera insérée en premier, tout en haut dans la liste.
- **-R chain rulenum** : remplace la règle n° rulenum dans la chain indiquée.
- **-L** : liste les règles (nous l'avons déjà vu).
- **-F chain** : vide toutes les règles de la chain indiquée. Cela revient à supprimer toutes les règles une par une pour cette chain.
- **-P chain regle** : modifie la règle par défaut pour la chain. Cela permet de dire, par exemple, que par défaut tous les ports sont fermés, sauf ceux que l'on a indiqués dans les règles. De manière générale, l'ajout d'une règle se passe suivant ce schéma :

iptables -A (chain) -p (protocole) --dport (port) -j (dé cision)

Remplacez chain par la section qui vous intéresse (INPUT ou OUTPUT), protocole par le nom du protocole à filtrer (TCP, UDP, ICMP. . .) et enfin décision par la décision à prendre : ACCEPT pour accepter le paquet, REJECT pour le rejeter ou bien DROP pour l'ignorer complètement. Le mieux est de découvrir comment on ajoute une règle par une série d'exemples.

```
# iptables -A INPUT -p tcp --dport ssh -j ACCEPT
```

Cela ajoute à la section INPUT (donc, pour le trafic entrant) une règle sur les données reçues via le protocole TCP sur le port de ssh (vous pouvez remplacer ssh par le numéro du port, soit 22). Lorsque votre ordinateur recevra des données en TCP sur le port de SSH, celles-ci seront acceptées ; cela vous permettra donc de vous connecter à distance à votre PC via SSH.

Vous pouvez faire de même avec d'autres ports :

```
# iptables -A INPUT -p tcp --dport www -j ACCEPT
```

. . . pour le web (80).

```
# iptables -A INPUT -p tcp --dport imap2 -j ACCEPT
```

. . . pour les mails, etc.

Si vous ne précisez pas de port (en omettant la section dport), tous les ports seront acceptés !

10.4 Autoriser les pings

En plus d'autoriser le trafic sur ces ports, je peux vous conseiller d'autoriser le protocole ICMP (pour pouvoir faire un ping) sur tous ces derniers :

```
# iptables -A INPUT -p icmp -j ACCEPT
```

Comme je n'ai pas indiqué de section –dport, cette règle s'applique à tous les ports, **mais pour les pings (icmp) uniquement** ! Votre ordinateur répondra alors aux « pings » pour indiquer qu'il est bien en vie.

10.5 Autoriser les connexions locales et déjà ouvertes

Pour l'instant, nos règles sont encore un peu trop restrictives et pas vraiment utilisables (vous risquez de ne plus pouvoir faire grand-chose). Je vous propose d'ajouter deux règles pour « assouplir » un peu votre pare-feu et le rendre enfin utilisable.

```
# iptables -A INPUT -i lo -j ACCEPT
```

```
# iptables -A INPUT -m state --state ESTABLISHED , RELATED -j ACCEPT
```

Ces deux règles utilisent des options un peu différentes de celles que nous avons vues jusqu'ici. Voici quelques explications.

1. La première règle autorise tout le trafic sur l'interface de loopback locale grâce à -i lo. Il n'y a pas de risque à autoriser votre ordinateur à communiquer avec lui-même, d'autant plus qu'il en a parfois besoin !
2. La seconde règle autorise toutes les connexions qui sont déjà à l'état ESTABLISHED ou RELATED. En clair, elle autorise toutes les connexions qui ont été demandées par votre PC. Là encore, cela permet d'assouplir le pare-feu et de le rendre fonctionnel pour une utilisation quotidienne.

10.6 Refuser toutes les autres connexions par défaut

Il reste un point essentiel à traiter car, pour l'instant, ce filtrage ne sert à rien. En effet, nous avons indiqué quelles données nous autorisons, mais nous n'avons pas dit que toutes les autres devaient être refusées ! Changez donc la règle par défaut pour DROP par exemple :

```
# iptables -P INPUT DROP
```

Le filtrage est radical. Nous n'avons pas autorisé beaucoup de ports et il se pourrait que vous vous rendiez compte que certaines applications n'arrivent plus à accéder à l'internet (normal, leur port doit être filtré). À vous de savoir quels ports ces applications utilisent pour modifier les règles en conséquence. Au besoin, pensez à faire de même pour les règles de sortie (OUTPUT).

CHAPITRE 11

SERVICES RESEAUX ET DEPANNAGE

11.1 Configuration

11.1.1 Machines de type Debian

Le fichier de configuration des interfaces réseaux sous Debian est situé dans */etc/network/interfaces*.

```
# cat interfaces
auto lo eth0 eth1
iface lo inet loopback

iface eth0 inet static
    address 192.161.1.60
    netmask 255.255.255.0
    broadcast 192.168.1.255
    gateway 192.168.1.1

iface eth1 inet dhcp
```

Cet exemple montre trois types d'interfaces :

- l'interface lo de loopback,
- l'interface eth1 en dhcp, ne nécessitant pas de configuration plus avancée,
- l'interface eth0 configurée statiquement.

La syntaxe générale d'une déclaration est la suivante :

interface nom type mode

Avec une configuration statique, précisez les différents paramètres avec les mots clés suivants :

- **address** : l'adresse IP.
- **netmask** : le masque de sous-réseau.
- **broadcast** : l'adresse de broadcast.
- **gateway** : la passerelle par défaut.

Le fichier **/etc/hostname** contient le nom de la machine :

```
# cat /etc/hostname
slyserver
```

11.2. Les services

Le fichier **/etc/services** contient la liste des services réseaux connus de Unix ainsi que les ports et protocoles associés. Il est utilisé par de nombreux services (dont xinetd) et sous-systèmes comme le firewall de Linux. Ce fichier est indicatif : c'est un fichier de description et de définition : tous les services de ce fichier ne tournent pas forcément sur votre machine (et heureusement vu le nombre) ! Et un service peut être configuré

pour écouter un autre port. Par contre il est conseillé, lorsque vous rajoutez un service qui n'est pas présent dans ce fichier, de le rajouter à la fin.

11.3 Les protocols: */etc/protocols*

Le fichier */etc/protocols* contient la liste des protocoles connus par Unix.

11.4 Services réseaux *xinetd*

Le démon **xinetd** est un « super-service » permettant de contrôler l'accès à un ensemble de services, **telnet** par exemple. Beaucoup de services réseaux peuvent être configurés pour fonctionner avec *xinetd*, comme les services ftp, ssh, samba, http, etc. Des options de configuration spécifiques peuvent être appliquées pour chaque service géré.

11.5 Monter un serveur DHCP

11.5.1. Présentation

Le service **DHCP** (*Dynamic Host Configuration Protocol*), protocole de configuration dynamique des hôtes, permet aux hôtes d'un réseau de demander et recevoir des informations de configuration (adresse, routage, DNS, etc.). Il y a en général un seul serveur DHCP par segment de réseau même si plusieurs sont possibles. Si le serveur est sur un autre segment, on peut utiliser un agent de retransmission DHCP. Autrement dit, un client DHCP recherche tout seul un serveur DHCP qui lui communiquera son adresse IP. L'adresse IP est assignée soit dynamiquement à partir de plages d'adresses prédéfinies, soit statiquement en fonction de l'adresse MAC du demandeur. Les informations sont valables un laps de temps donné (un bail) qui peut être renouvelé et configurable.

11.5.2. Serveur *dhcpcd*

a. Démarrage

Le serveur **dhcpcd** est un service (daemon) lancé à l'aide d'un script (*/etc/init.d/dhcpcd*). Il est configuré à l'aide du fichier */etc/dhcp/dhcpclient.conf*. Les adresses IP allouées sont placées dans */var/lib/dhcp/dhcpcd.leases*.

service dhcpcd start Ou : *# /etc/init.d/dhcpcd start*

Informations de base

```
ddns-update-style none; # pas de mise à jour du DNS par DHCP
option domain-name "toto.fr"; # nom de domaine transmis au client
option domain-name-servers 192.168.1.254; # liste des DNS séparés
par des virgules
default-lease-time 21600; # durée du bail par défaut en secondes
sans demande explicite
max-lease-time 43200; # durée max du bail si la demande du client
est plus élevée
```

Comme `dhcpd` peut gérer plusieurs sous-réseaux, on doit lui préciser les règles à appliquer pour chaque sous-réseau. Généralement dans le cadre d'un petit réseau un seul bloc sera présent mais tous les cas sont envisageables. Si vous êtes certain de n'avoir qu'un seul réseau, vous pouvez omettre la déclaration du subnet (sous-réseau).

```
# Gestion du sous-réseau 192.168.1.0
subnet 192.168.1.0 netmask 255.255.255.0
{
    option routers 192.168.1.254; # passerelle pour ce réseau
    option subnet-mask 255.255.255.0; # masque de sous-réseau

range 192.168.1.2 192.168.1.250; # Configuration de l'intervalle DHCP

# Cas d'attributions d'IP statiques
host station1
{
    hardware ethernet 00:A0:ad:41:5c:b1; # Adresse MAC
    fixed-address 192.168.1.1; # cette machine aura l'IP 192.168.1.1
}
```

Certains clients DHCP ignorent totalement le fait qu'un serveur DHCP peut dynamiquement allouer un nom (hostname) à l'hôte. Dans l'exemple précédent, la machine avec l'IP 192.168.1.1 devrait obtenir le nom `station1`. Voici un exemple :

```
# les hôtes se verront attribués les noms des host déclarés
use-host-decl-names on;
host station1
{
    hardware ethernet 00:A0:ad:41:5c:b1; # Adresse MAC
    fixed-address 192.168.1.1; # cette machine aura l'IP 192.168.1.1
    et le nom station1
}
```

Vous pouvez aussi travailler au cas par cas :

```
host station2
{
    hardware ethernet 00:A0:ad:41:5c:b2; # Adresse MAC
    fixed-address 192.168.1.251; # ce host aura l'IP 192.168.1.251
    option host-name "station2"; # ce host aura comme nom station2
}
```

b. Côté client

Sous Linux et les distributions de type Red Hat, Fedora, Mandriva, openSUSE, etc., modifiez le fichier `/etc/sysconfig/network-script/ifcfg-xxx` en spécifiant `dhcp` pour **BOOTPROTO** et relancez la connexion réseau (`ifdown` puis `ifup`). Le client `dhcpcd`

permet d'activer dhcp sur une interface réseau. La méthode la plus simple est, par exemple pour eth0 :

```
# dhcpcd eth0 &
```

Vous pouvez transmettre des options à dhcpcd pour prendre en charge diverses possibilités. Parmi ces options :

- -D : autorise la modification du nom de domaine
- -H : autorise la modification du nom d'hôte
- -R : évite l'écrasement du fichier resolv.conf
- -l : permet de modifier le leasetime (en secondes).

```
# dhcpcd -D -H -l 86400 eth0
```

11.6 Serveur DNS

11.6.1. Présentation

Le Système de Noms de Domaines **DNS** (*Domain Name System*) transforme les noms d'hôte en adresses IP : c'est la **résolution de nom**. Il transforme les adresses IP en noms d'hôte : c'est la **résolution inverse**. Il permet de regrouper les machines par domaines de nom. Il fournit des informations de routage et de courrier électronique.

Le DNS permet de faire référence à des systèmes basés sur IP (les *hôtes*) à l'aide de noms conviviaux (les *noms de domaines*). L'intérêt d'un DNS est évident. Les noms de domaine sont plus simples à retenir, et si son adresse IP change l'utilisateur ne s'en rend même pas compte. On comprend que le DNS est un service clé critique pour Internet. Les noms de domaine sont séparés par des points, chaque élément pouvant être composé de 63 caractères il ne peut y avoir qu'un maximum de 127 éléments et le nom complet ne doit pas dépasser 255 caractères. Le nom complet non abrégé est appelé **FQDN** (*Fully Qualified Domain Name*). Dans un FQDN, l'élément le plus à droite est appelé **TLD** (*Top Level Domain*), celui le plus à gauche représente l'hôte et donc l'adresse IP.

11.6.2. Lancement

Le service s'appelle **named**.

```
# service named start Ou # /etc/init.d/named start
```

11.6.3. Configuration de Bind

Bind (*Berkeley Internet Name Daemon*) est le serveur de noms le plus utilisé sur Internet. Bind 9 supporte l'IPv6, les noms de domaine unicode, le multithread et de nombreuses améliorations de sécurité.

a. Configuration générale

La configuration globale de Bind est placée dans le fichier **/etc/named.conf**. La configuration détaillée des zones est placée dans **/var/lib/named**. **/etc/named.conf** est composé de deux parties. La première concerne la configuration globale des options de Bind. La seconde est la déclaration des zones pour les domaines individuels. Les commentaires commencent par un **#** ou **//**.

```
# cat /etc/sysconfig/named
...
CHROOT=/var/named/chroot
...
```

Dans ce cas, tous les fichiers de configuration, y compris les zones, sont relatifs à ce chemin. Voici un fichier **named.conf** de base.

```
options {
    directory "/var/lib/named";
    forwarders { 10.0.0.1; };
    notify no;
};
zone "localhost" in {
    type master;
    file "localhost.zone";
};
zone "0.0.127.in-addr.arpa" in {
    type master;
    file "127.0.0.zone";
};
zone "." in {
    type hint;
    file "root.hint";
};
```

11.7 Service HTTP Apache

11.7.1. Présentation

Apache 2 est le serveur HTTP le plus utilisé actuellement sur les serveurs Web. Sa configuration et sa flexibilité en font un serveur incontournable.

Lorsqu'un serveur Apache reçoit des requêtes, il peut les redistribuer à des processus fils. La configuration permet de lancer des processus de manière anticipée et d'adapter dynamiquement ce nombre en fonction de la charge.

Apache est modulaire. Chaque module permet d'ajouter des fonctionnalités au serveur. Le module le plus connu est probablement celui gérant le langage PHP, « **mod_php** ». Chaque module s'ajoute via les fichiers de configuration, et il n'y a même pas besoin de relancer le serveur Apache : on lui donne juste l'ordre de relire sa configuration.

Apache peut gérer plusieurs sites Web en même temps, ayant chacun leur nom, à l'aide des hôtes virtuels.

11.7.2. Arrêt/Relance

Le nom du service dépend de la distribution. Il est souvent intitulé **apache** ou **httpd**. Suivant la distribution lancez le service via la commande **service** ou directement par son nom `/etc/init.d/apache`.

- **/etc/init.d/httpd start** : démarre
- **/etc/init.d/httpd stop** : stoppe
- **/etc/init.d/httpd restart** : redémarre
- **/etc/init.d/httpd reload** : demande à Apache de relire sa configuration sans redémarrer.

Apache est fourni avec l'outil **apachectl** qui reprend les paramètres (liste non exhaustive) `start`, `stop`, `status`, `reload`, et surtout **configtest** qui valide ou non le contenu du fichier de configuration de Apache.

11.7.3. Configuration

La configuration principale est stockée dans `/etc/httpd/conf/httpd.conf`. Elle contrôle les paramètres généraux du serveur Web, les hôtes virtuels et les accès. La configuration des différents modules est placée dans `/etc/httpd/conf.d`. Les modules sont présents dans `/etc/httpd/modules/`. Par défaut la racine du serveur, celle où sont placées les pages du site, est dans `/var/www` ou `/srv/www`. Cette position dépend de la directive **DocumentRoot** dans les fichiers de configuration.

11.8 Problèmes de maintenance

L'administration d'un système Linux comporte plusieurs aléas revenant de façon récurrente. À la manière d'une FAQ (*Frequently Asked Questions*), genre popularisé sur Internet pour apporter les réponses aux utilisateurs, voici une liste de questions et de leurs réponses.

1. "Sur quelle version Ubuntu suisje exactement ?"

De cette question anodine va découler deux réponses. La première renvoie l'identification du noyau en cours, du type de version et de quelques informations complémentaires : `uname -r`. Exemple :

```
Linux srvfrw 2.6.24-19-server #1 SMP Wed Jun 18 15:18:00 UTC 2008 i686 GNU/Linux
```

L'option `-a` renvoie une version réduite. Cette information est utile car le système peut contenir plusieurs noyaux, présents au démarrage dans l'écran de GRUB, et

l'administrateur évoluant entre l'un ou l'autre en fonction des besoins (choix différents de modules par exemple). Pour la demande d'identification de la version, la commande s'énonce comme : `lsb_release -a`

2. "J'ai perdu mon mot de passe administrateur..."

L'être humain est faillible, l'administrateur système est un être humain donc un administrateur est faillible. Ce syllogisme facile montre qu'il existe des défaillances quel que soit le sérieux et la compétence de l'administrateur. ■ À partir d'un CDRom de la distribution Ubuntu serveur et en redémarrant la machine, choisissez l'option **Réparer un système endommagé**. Après la question classique du choix de la langue, suit une série de détections de la configuration puis une demande sur le nom de la machine (facultatif, car sans effet pour la suite). Le cheminement du mode de récupération aboutit enfin à un écran de choix du système racine.

Exemple pour un système avec trois partitions : /boot, swap et /

```

[!!!] Entrer dans le mode de récupération

Veuillez choisir un périphérique qui sera utilisé comme système de
fichiers racine. Vous pourrez ensuite choisir entre plusieurs
opérations de récupération à effectuer sur ce système de fichiers.

Périphérique à monter comme système de fichiers racine (/) :

/dev/sda1
/dev/sda2
/dev/sda3

<Revenir en arrière>
```

L'administrateur doit savoir où se situe le montage du système de fichiers racine (dans l'exemple /dev/sda3). S'en suivent les options cidessous :

```

[!!!] Entrer dans le mode de récupération

Opérations du mode de récupération

Exécuter un shell dans /dev/sda3
Exécuter un shell dans le contexte de l'installateur
Réinstallation du programme de démarrage GRUB
Changer de système de fichiers racine
Redémarrer le système

<Revenir en arrière>
```

On s'aperçoit que la réinstallation de GRUB est possible en cas de problème de cet ordre. Pour changer un mot de passe nous avons besoin d'un shell aussi est-ce le premier choix à prendre. Après l'information sur le montage du système, un prompt

(invite de commande) s'offre en root pour lancer la commande `passwd` afin de changer le mot de passe. Il ne reste plus alors qu'à sortir du montage par un simple **exit**.

La relative simplicité de passer outre une règle de sécurité essentielle comme le changement de mot de passe de l'administrateur, met en exergue la nécessité absolue de protéger physiquement les accès aux serveurs...

3. "Le serveur graphique ne répond plus..."

Cas typique d'un problème graphique, il arrive que la souris ne réponde plus, ni le clavier. Xorg porte sa part avec son constant développement et sa constitution modulaire (version 7.3 sur Hardy Heron) et les pilotes propriétaires de cartes graphiques, une autre... Quoi qu'il en soit, une simple combinaison de touches permet d'arrêter le serveur graphique et de revenir :

- soit en mode console si le serveur a été lancé par la commande `startx`
- soit sur GDM en poste de travail. `[Ctrl][Alt][FlècheRetour]`

Note : la touche [Retour arrière] ou *backspace* indique la touche unique avec une flèche gauche généralement en haut à droite du clavier et non la flèche de direction.

4. "J'ai un programme qui bloque, impossible de l'arrêter..."

Il suffit de "tuer" son processus. Cette opération s'effectue en ligne de commande avec les droits administrateurs

5. "La console n'est pas/plus en français..."

Accessoirement, vous avez bien un clavier **azerty** mais pas les caractères accentués. Cela arrive parfois lorsque la configuration initiale n'a pas été correctement terminée.

Le plus simple est de faire un : `dpkg-reconfigure console-setup`

Cette relance de la configuration écrit un nouveau fichier `/etc/default/console-setup` qui, pour les français ressemble à ceci :

```
VERBOSE_OUTPUT=no
ACTIVE_CONSOLES="/dev/tty[1-6]"
CHARMAP="UTF-8"
CODESET="Lat15"
FONTFACE="VGA"
FONTSIZE="16"
XKBMODEL="pc105"
XKBLAYOUT="fr"
XKBVARIANT="oss"
XKBOPTIONS="lv3:ralt_switch"
BOOTTIME_KMAP_MD5="aaaaaaaaaaaaaaaaaaaaaaaaaaaa"
```